

```
In [1]: from rdflib import Graph
        from collections import defaultdict, Counter

        # Load the RDF data from the file
        rdf_file = "results.ttl"
        g = Graph()
        g.parse(rdf_file, format="turtle")

        # Dictionary to hold property -> Counter of values
        property_value_counts = defaultdict(Counter)

        # Iterate through all triples and count values for each property
        for s, p, o in g:
            property_value_counts[p][o] += 1

        # Print duplicate values for each property
        for prop, values in property_value_counts.items():
            duplicates = {val: count for val, count in values.items() if count > 1}
            print(f"\nProperty: {prop}")
            if duplicates:
                print("  Duplicate values:")
                for val, count in duplicates.items():
                    print(f"    {val} (Count: {count})")
            else:
                print("  No duplicates.")
```

Property: <http://example.org/hasCountry>

Duplicate values:

- <https://dbpedia.org/page/Japan> (Count: 12)
- <https://dbpedia.org/page/France> (Count: 8)
- <https://dbpedia.org/page/India> (Count: 15)
- <https://dbpedia.org/page/Nigeria> (Count: 12)
- <https://dbpedia.org/page/Brazil> (Count: 9)
- <https://dbpedia.org/page/USA> (Count: 3)

Property: <http://example.org/hasStreet>

No duplicates.

Property: <http://example.org/isAllergicTo>

Duplicate values:

- https://dbpedia.org/page/Peanut_allergy (Count: 16)
- https://dbpedia.org/page/Soy_allergy (Count: 19)
- https://dbpedia.org/page/Lactose_intolerance (Count: 14)
- https://dbpedia.org/page/Gluten_allergy (Count: 16)
- https://dbpedia.org/page/Dust_mite_allergy (Count: 3)
- https://dbpedia.org/page/Pollen_allergy (Count: 3)

Property: <http://example.org/spokenIn>

Duplicate values:

- <https://dbpedia.org/page/Spain> (Count: 3)
- <https://dbpedia.org/page/USA> (Count: 9)
- <https://dbpedia.org/page/France> (Count: 19)
- <https://dbpedia.org/page/India> (Count: 10)
- <https://dbpedia.org/page/Nigeria> (Count: 12)
- <https://dbpedia.org/page/Japan> (Count: 10)
- <https://dbpedia.org/page/Brazil> (Count: 10)
- <https://dbpedia.org/page/UK> (Count: 2)
- <https://dbpedia.org/page/China> (Count: 2)
- <https://dbpedia.org/page/Italy> (Count: 6)

Property: <http://example.org/hasPostalCode>

No duplicates.

Property: <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>

Duplicate values:

- <http://example.org/Person> (Count: 60)

Property: <http://example.org/belongsToHospitalSection>

Duplicate values:

- <https://www.wikidata.org/wiki/Q56857410> (Count: 21)
- <https://www.wikidata.org/wiki/Q56857411> (Count: 29)
- <https://www.wikidata.org/wiki/Q56857412> (Count: 8)
- <https://www.wikidata.org/wiki/Q56857413> (Count: 2)

Property: <http://example.org/hasAge>

Duplicate values:

- 75 (Count: 2)
- 68 (Count: 3)
- 76 (Count: 2)
- 69 (Count: 5)
- 22 (Count: 2)
- 56 (Count: 2)

34 (Count: 2)
65 (Count: 2)
71 (Count: 2)
74 (Count: 3)
61 (Count: 4)
63 (Count: 3)
29 (Count: 3)
48 (Count: 2)
42 (Count: 2)

Property: <http://example.org/hasEducationLevel>

Duplicate values:

<https://www.wikidata.org/wiki/Q163727> (Count: 21)
<https://www.wikidata.org/wiki/Q183816> (Count: 19)
<https://www.wikidata.org/wiki/Q849697> (Count: 20)

Property: <http://example.org/hasMaritalStatus>

Duplicate values:

<https://www.wikidata.org/wiki/Q179115> (Count: 23)
<https://www.wikidata.org/wiki/Q28341938> (Count: 23)
<https://www.wikidata.org/wiki/Q37713312> (Count: 14)

Property: <http://example.org/hasIncome>

No duplicates.

Property: <http://example.org/hasDiseaseDiagnosis>

Duplicate values:

<https://dbpedia.org/page/Malaria> (Count: 10)
https://dbpedia.org/page/Diabetes_mellitus (Count: 15)
<https://dbpedia.org/page/Tuberculosis> (Count: 7)
<https://dbpedia.org/page/Hypertension> (Count: 19)
<https://dbpedia.org/page/Asthma> (Count: 12)

Property: <http://example.org/hasHairColor>

Duplicate values:

Gray (Count: 11)
Red (Count: 9)
Black (Count: 14)
Blonde (Count: 13)
Brown (Count: 13)

Property: <http://example.org/takesMedication>

Duplicate values:

<https://dbpedia.org/page/Isoniazid> (Count: 14)
<https://dbpedia.org/page/Lisinopril> (Count: 10)
<https://dbpedia.org/page/Metformin> (Count: 12)
<https://dbpedia.org/page/Artemisinin> (Count: 14)
<https://dbpedia.org/page/Salbutamol> (Count: 9)

Property: <http://example.org/hasName>

No duplicates.

Property: <http://example.org/hasCity>

Duplicate values:

<https://dbpedia.org/page/Tokyo> (Count: 17)
<https://dbpedia.org/page/Paris> (Count: 12)

<https://dbpedia.org/page/Mumbai> (Count: 11)
<https://dbpedia.org/page/Lagos> (Count: 12)
https://dbpedia.org/page/Sao_Paulo (Count: 7)

Property: <http://example.org/hasGeneralPractice>
Duplicate values:
Alexander-Palmer (Count: 2)

Property: <http://example.org/hasBloodType>
Duplicate values:
<https://www.wikidata.org/wiki/Q19831454> (Count: 15)
<https://www.wikidata.org/wiki/Q19831449> (Count: 17)
<https://www.wikidata.org/wiki/Q19831455> (Count: 13)
<https://www.wikidata.org/wiki/Q19831453> (Count: 15)

```
In [2]: from rdflib import Graph
        from collections import defaultdict, Counter
        import matplotlib.pyplot as plt

        # Load RDF graph from file
        rdf_file = "results.ttl"
        g = Graph()
        g.parse(rdf_file, format="turtle")

        # Track value frequencies per property
        property_value_counts = defaultdict(Counter)

        # Count all values for each property
        for s, p, o in g:
            property_value_counts[p][o] += 1

        # Prepare duplicate summary
        duplicate_summary = {}

        for prop, values in property_value_counts.items():
            duplicates = {val: count for val, count in values.items() if count > 1}
            if duplicates:
                duplicate_summary[str(prop)] = duplicates

        # Print summary
        for prop, dup_vals in duplicate_summary.items():
            print(f"\nProperty: {prop}")
            for val, count in dup_vals.items():
                print(f"  {val} (Count: {count})")

        # Visualization: Bar plot per property
        for prop, dup_vals in duplicate_summary.items():
            plt.figure(figsize=(10, 4))
            labels = [str(v) for v in dup_vals.keys()]
            counts = list(dup_vals.values())

            plt.barh(labels, counts, color='skyblue')
            plt.xlabel("Frequency")
            plt.title(f"Duplicate Values for Property: {prop}")
            plt.tight_layout()
            plt.show()
```

Property: <http://example.org/hasCountry>
 <https://dbpedia.org/page/Japan> (Count: 12)
 <https://dbpedia.org/page/France> (Count: 8)
 <https://dbpedia.org/page/India> (Count: 15)
 <https://dbpedia.org/page/Nigeria> (Count: 12)
 <https://dbpedia.org/page/Brazil> (Count: 9)
 <https://dbpedia.org/page/USA> (Count: 3)

Property: <http://example.org/isAllergicTo>
 https://dbpedia.org/page/Peanut_allergy (Count: 16)
 https://dbpedia.org/page/Soy_allergy (Count: 19)
 https://dbpedia.org/page/Lactose_intolerance (Count: 14)
 https://dbpedia.org/page/Gluten_allergy (Count: 16)
 https://dbpedia.org/page/Dust_mite_allergy (Count: 3)
 https://dbpedia.org/page/Pollen_allergy (Count: 3)

Property: <http://example.org/spokenIn>
 <https://dbpedia.org/page/Spain> (Count: 3)
 <https://dbpedia.org/page/USA> (Count: 9)
 <https://dbpedia.org/page/France> (Count: 19)
 <https://dbpedia.org/page/India> (Count: 10)
 <https://dbpedia.org/page/Nigeria> (Count: 12)
 <https://dbpedia.org/page/Japan> (Count: 10)
 <https://dbpedia.org/page/Brazil> (Count: 10)
 <https://dbpedia.org/page/UK> (Count: 2)
 <https://dbpedia.org/page/China> (Count: 2)
 <https://dbpedia.org/page/Italy> (Count: 6)

Property: <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
 <http://example.org/Person> (Count: 60)

Property: <http://example.org/belongsToHospitalSection>
 <https://www.wikidata.org/wiki/Q56857410> (Count: 21)
 <https://www.wikidata.org/wiki/Q56857411> (Count: 29)
 <https://www.wikidata.org/wiki/Q56857412> (Count: 8)
 <https://www.wikidata.org/wiki/Q56857413> (Count: 2)

Property: <http://example.org/hasAge>
 75 (Count: 2)
 68 (Count: 3)
 76 (Count: 2)
 69 (Count: 5)
 22 (Count: 2)
 56 (Count: 2)
 34 (Count: 2)
 65 (Count: 2)
 71 (Count: 2)
 74 (Count: 3)
 61 (Count: 4)
 63 (Count: 3)
 29 (Count: 3)
 48 (Count: 2)
 42 (Count: 2)

Property: <http://example.org/hasEducationLevel>
 <https://www.wikidata.org/wiki/Q163727> (Count: 21)

<https://www.wikidata.org/wiki/Q183816> (Count: 19)
<https://www.wikidata.org/wiki/Q849697> (Count: 20)

Property: <http://example.org/hasMaritalStatus>
<https://www.wikidata.org/wiki/Q179115> (Count: 23)
<https://www.wikidata.org/wiki/Q28341938> (Count: 23)
<https://www.wikidata.org/wiki/Q37713312> (Count: 14)

Property: <http://example.org/hasDiseaseDiagnosis>
<https://dbpedia.org/page/Malaria> (Count: 10)
https://dbpedia.org/page/Diabetes_mellitus (Count: 15)
<https://dbpedia.org/page/Tuberculosis> (Count: 7)
<https://dbpedia.org/page/Hypertension> (Count: 19)
<https://dbpedia.org/page/Asthma> (Count: 12)

Property: <http://example.org/hasHairColor>
Gray (Count: 11)
Red (Count: 9)
Black (Count: 14)
Blonde (Count: 13)
Brown (Count: 13)

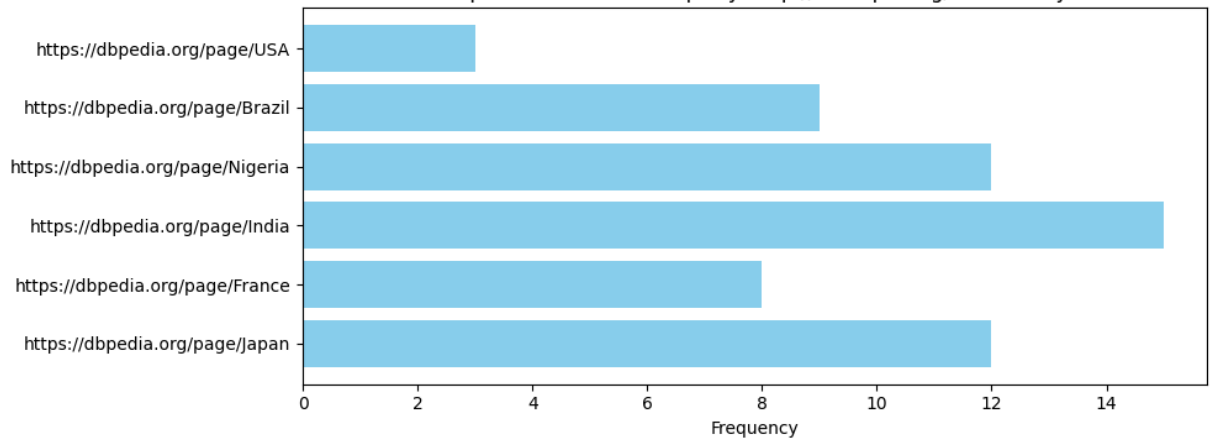
Property: <http://example.org/takesMedication>
<https://dbpedia.org/page/Isoniazid> (Count: 14)
<https://dbpedia.org/page/Lisinopril> (Count: 10)
<https://dbpedia.org/page/Metformin> (Count: 12)
<https://dbpedia.org/page/Artemisinin> (Count: 14)
<https://dbpedia.org/page/Salbutamol> (Count: 9)

Property: <http://example.org/hasCity>
<https://dbpedia.org/page/Tokyo> (Count: 17)
<https://dbpedia.org/page/Paris> (Count: 12)
<https://dbpedia.org/page/Mumbai> (Count: 11)
<https://dbpedia.org/page/Lagos> (Count: 12)
https://dbpedia.org/page/Sao_Paulo (Count: 7)

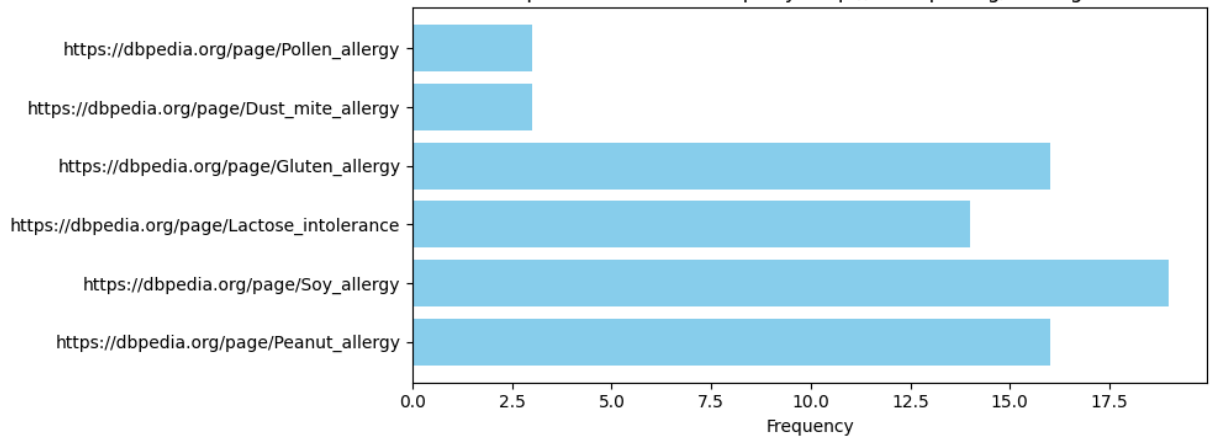
Property: <http://example.org/hasGeneralPractice>
Alexander-Palmer (Count: 2)

Property: <http://example.org/hasBloodType>
<https://www.wikidata.org/wiki/Q19831454> (Count: 15)
<https://www.wikidata.org/wiki/Q19831449> (Count: 17)
<https://www.wikidata.org/wiki/Q19831455> (Count: 13)
<https://www.wikidata.org/wiki/Q19831453> (Count: 15)

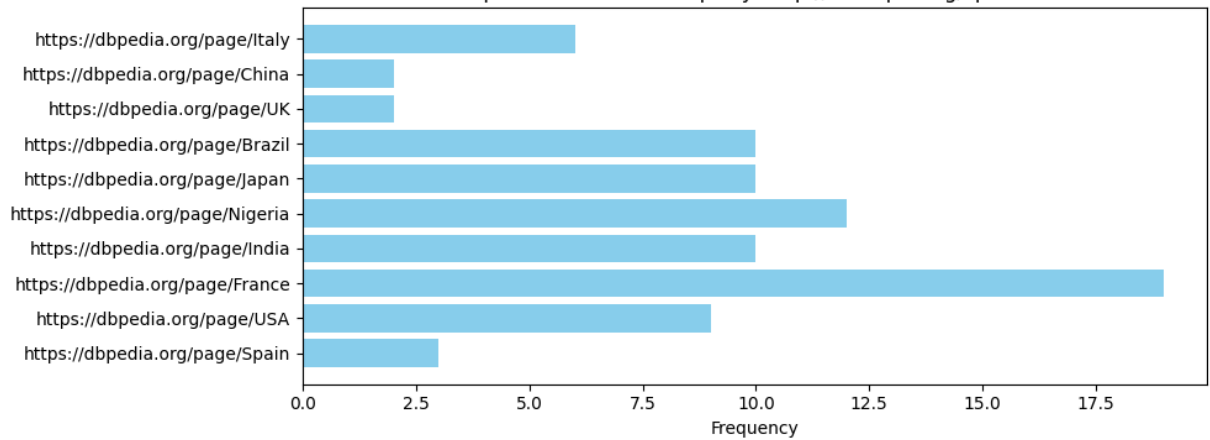
Duplicate Values for Property: http://example.org/hasCountry



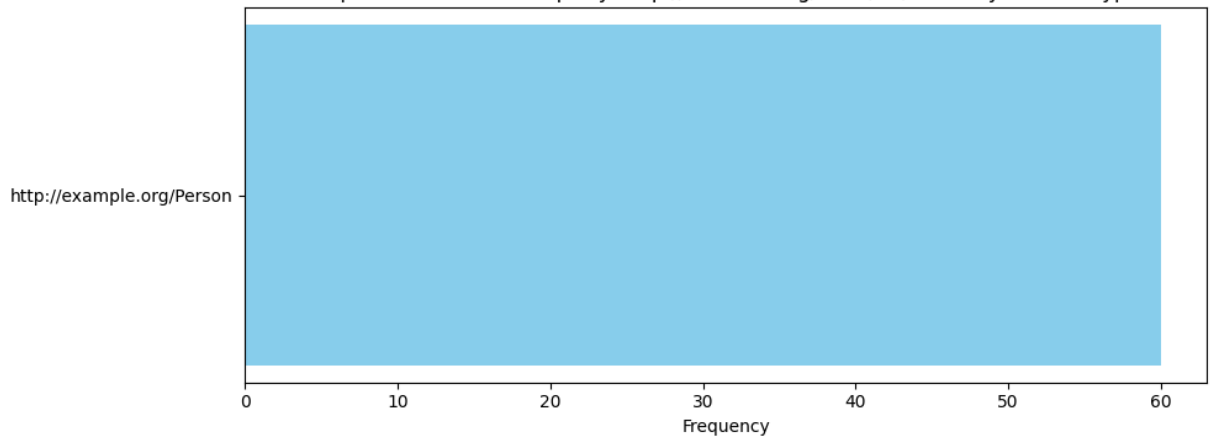
Duplicate Values for Property: http://example.org/isAllergicTo



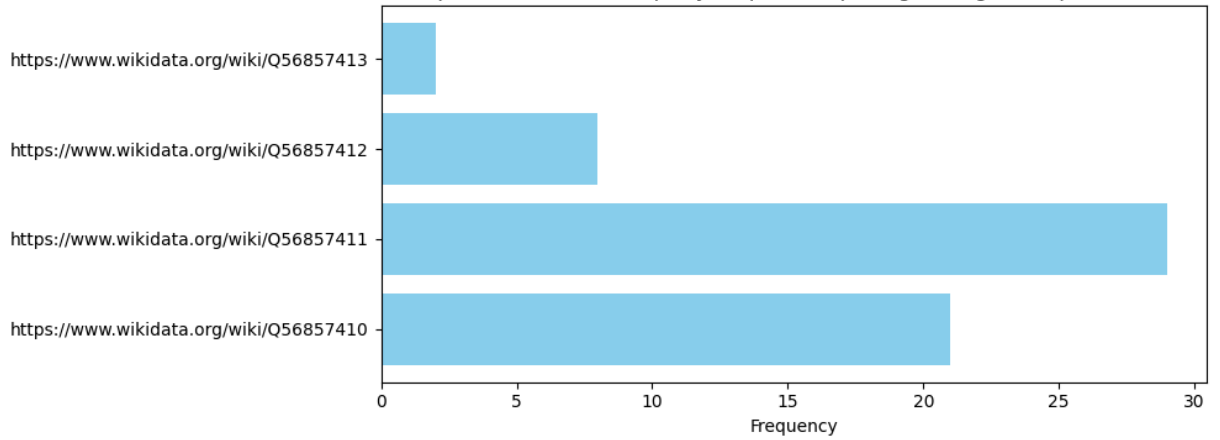
Duplicate Values for Property: http://example.org/spokenIn



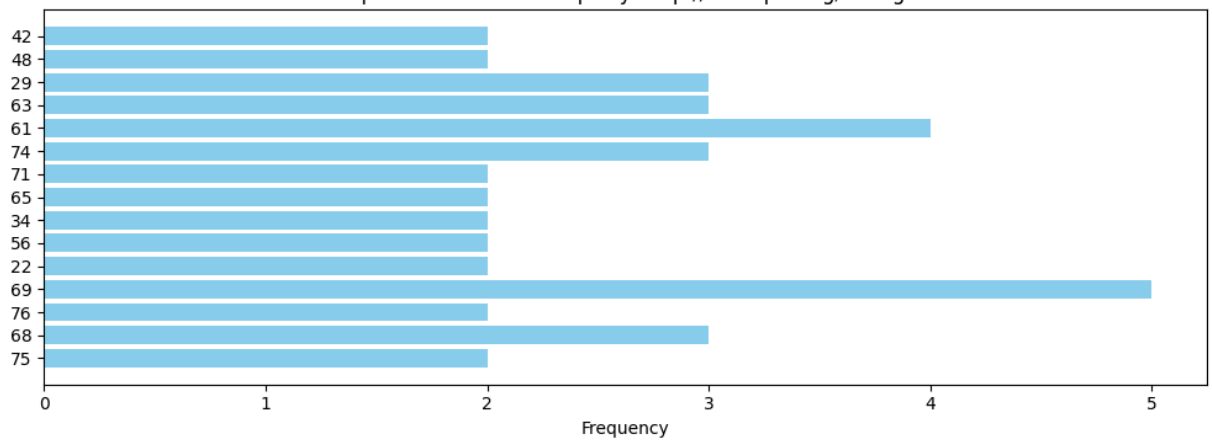
Duplicate Values for Property: <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>



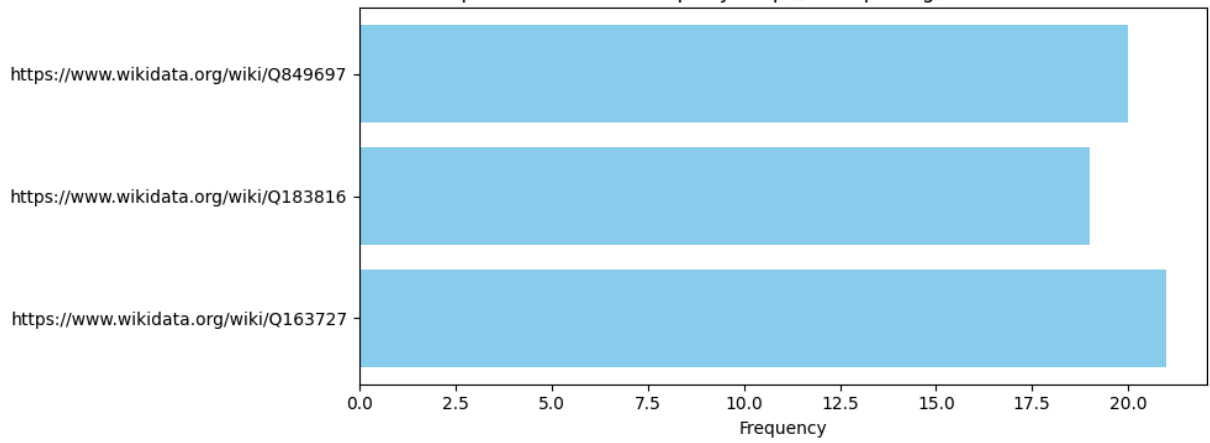
Duplicate Values for Property: <http://example.org/belongsToHospitalSection>



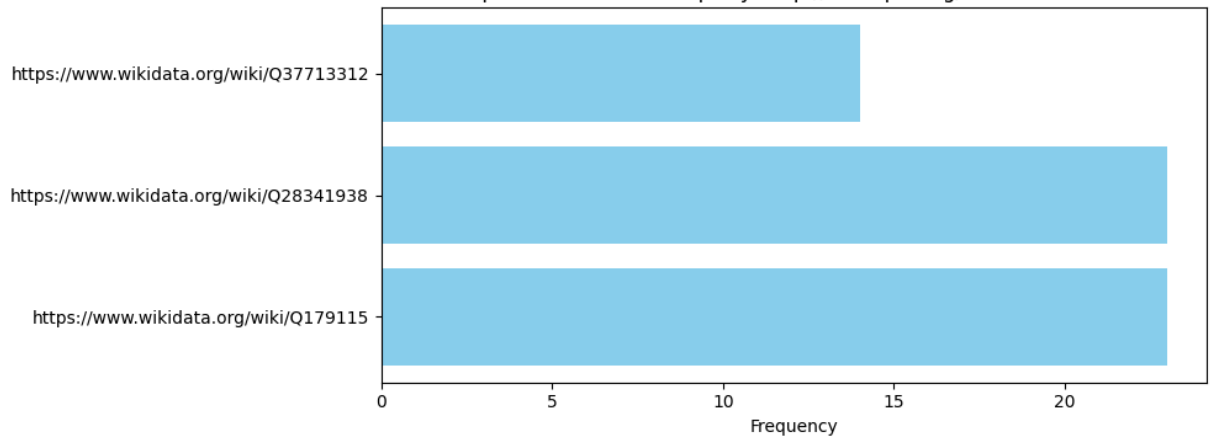
Duplicate Values for Property: <http://example.org/hasAge>



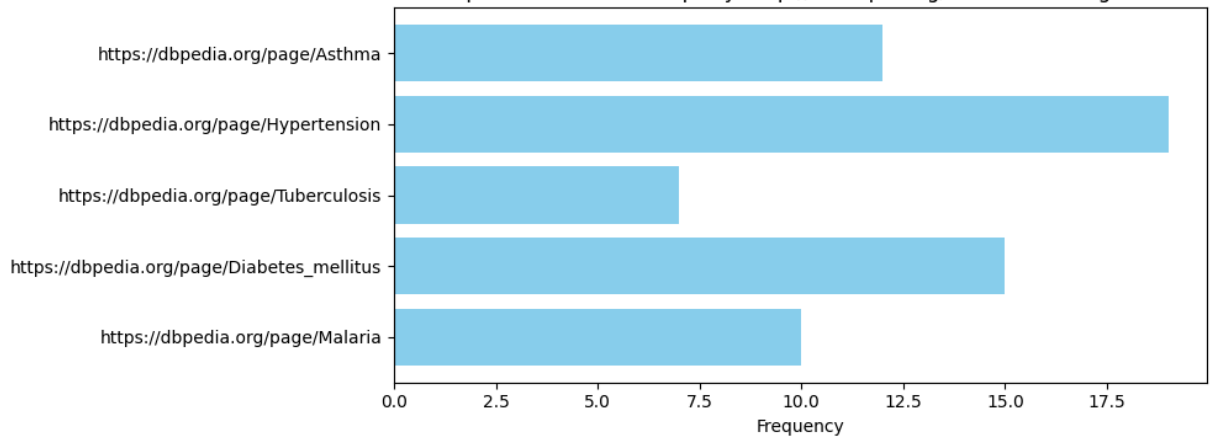
Duplicate Values for Property: <http://example.org/hasEducationLevel>

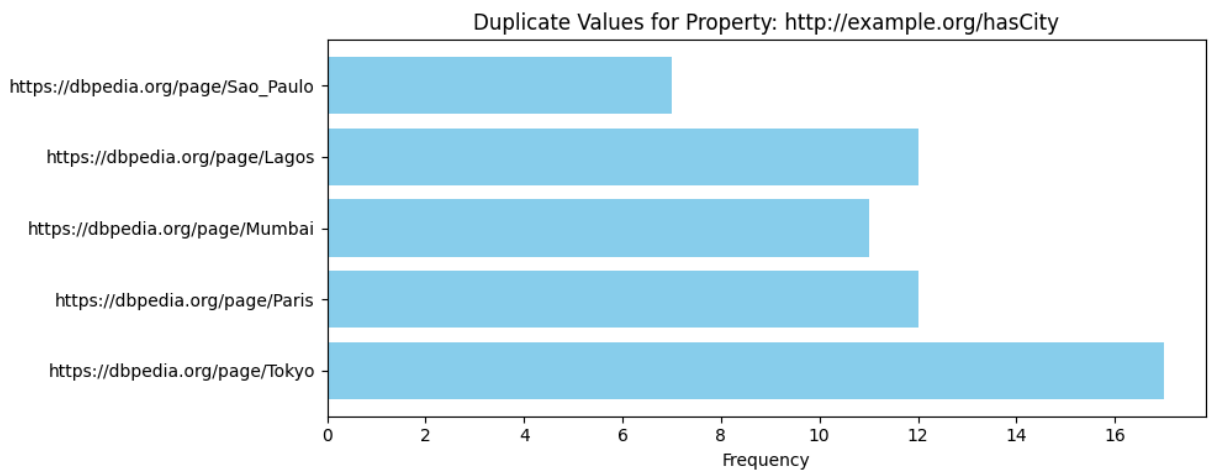
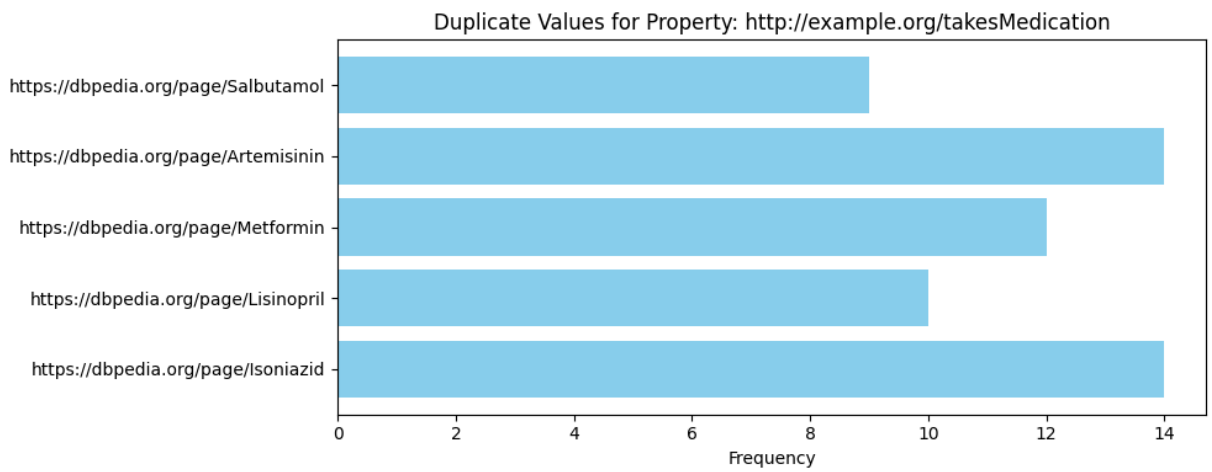
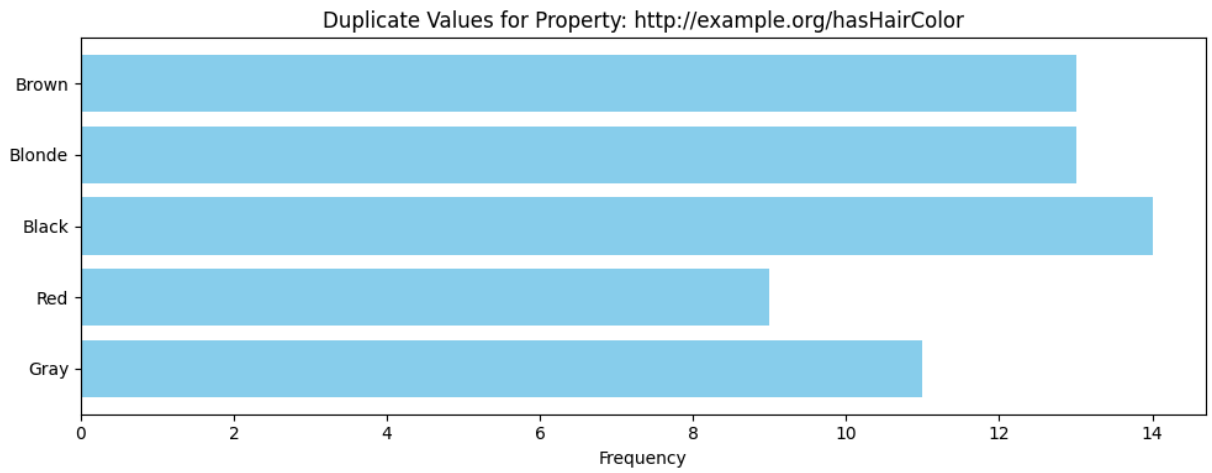


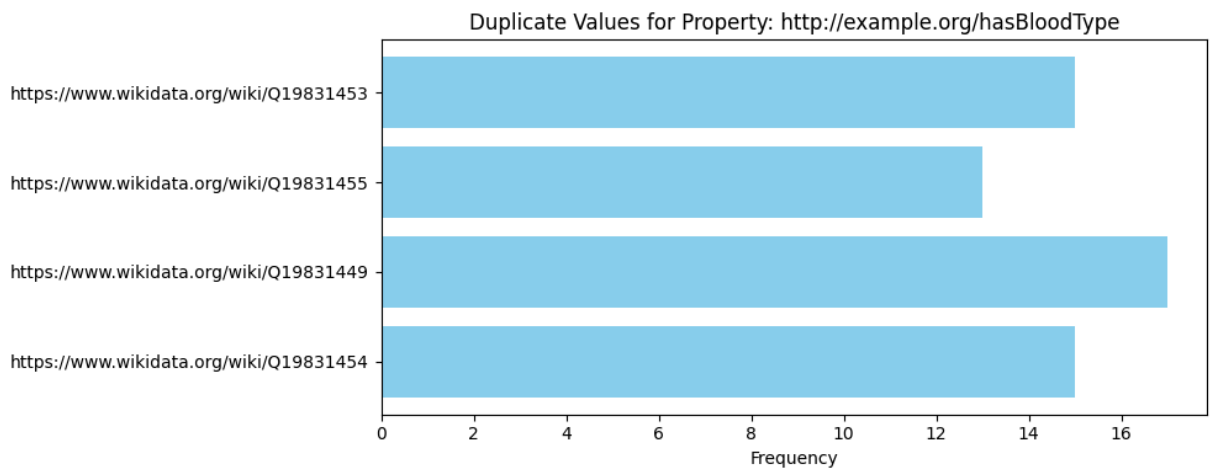
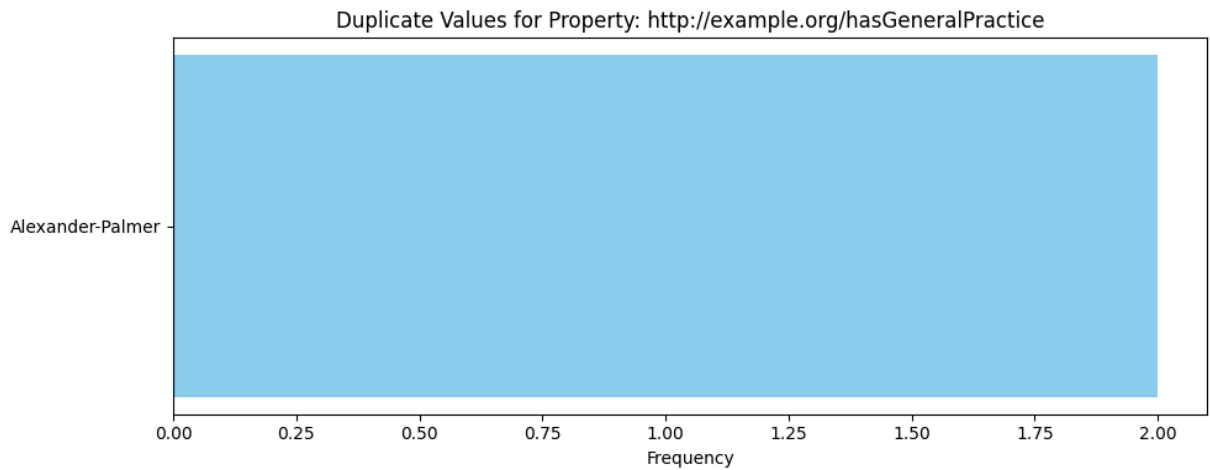
Duplicate Values for Property: <http://example.org/hasMaritalStatus>



Duplicate Values for Property: <http://example.org/hasDiseaseDiagnosis>







```
In [7]: import numpy as np
import matplotlib.cm as cm
import matplotlib.colors as mcolors

# Visualization: Bar plot per property with gradient color
for prop, dup_vals in duplicate_summary.items():
    plt.figure(figsize=(10, 4))
    labels = [str(v) for v in dup_vals.keys()]
    counts = list(dup_vals.values())

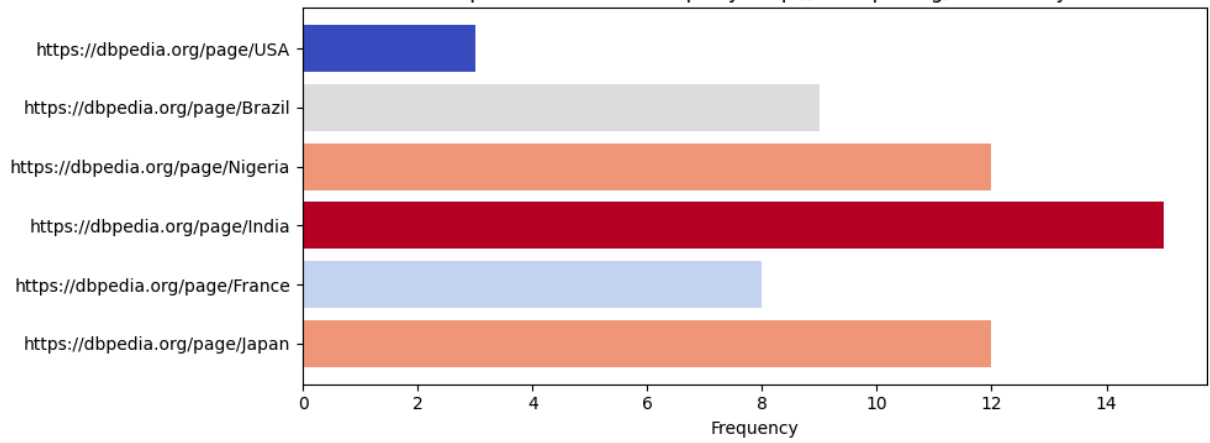
    # Normalize counts for color mapping
    norm = mcolors.Normalize(vmin=min(counts), vmax=max(counts))
    cmap = cm.get_cmap("coolwarm")
    colors = [cmap(norm(count)) for count in counts]

    plt.barh(labels, counts, color=colors)
    plt.xlabel("Frequency")
    plt.title(f"Duplicate Values for Property: {prop}")
    plt.tight_layout()
    plt.show()
```

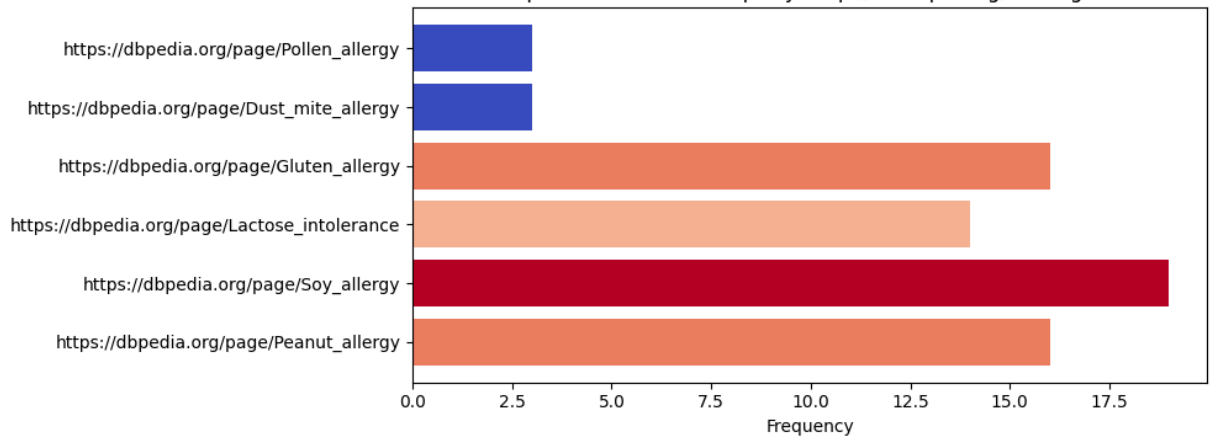
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/12922621.py:13: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

```
cmap = cm.get_cmap("coolwarm")
```

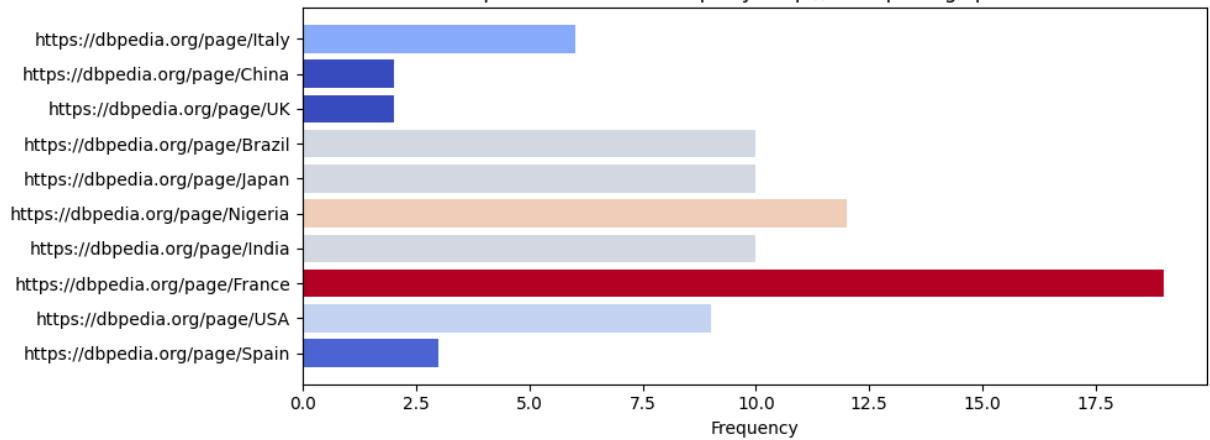
Duplicate Values for Property: <http://example.org/hasCountry>

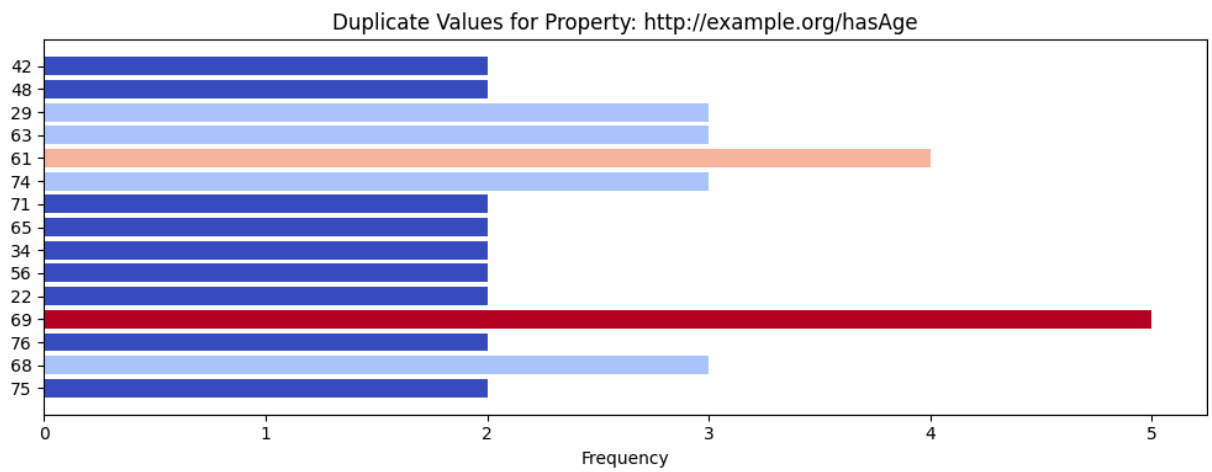
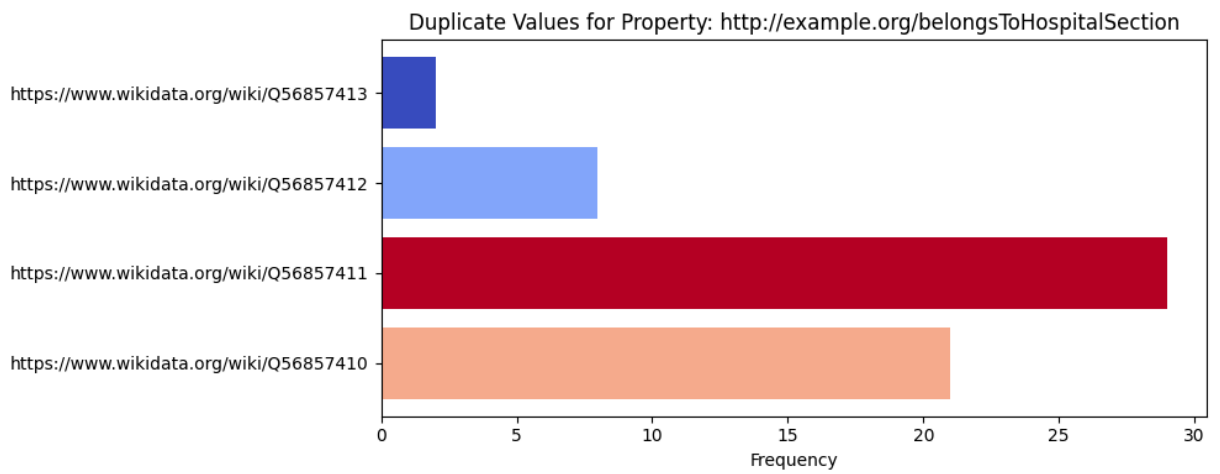
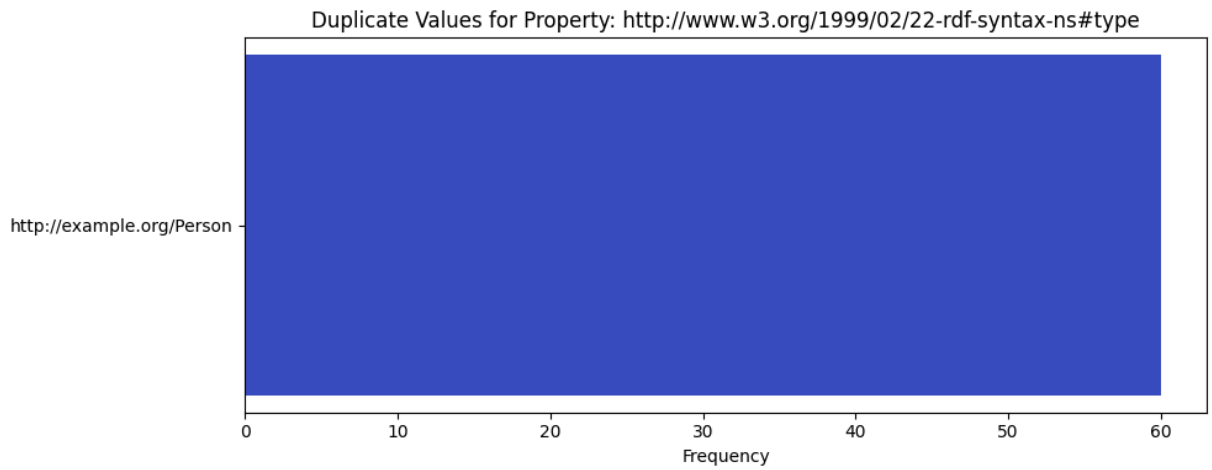


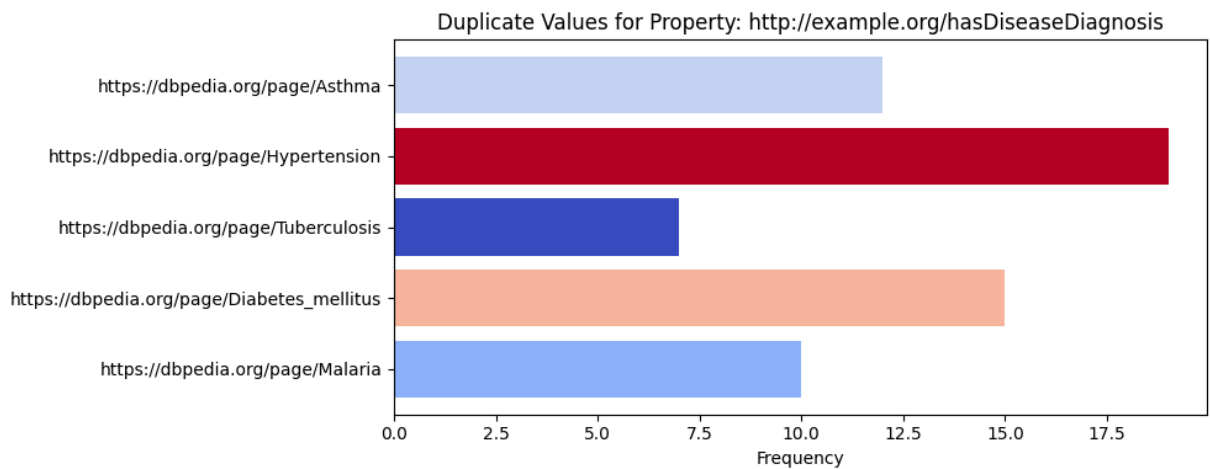
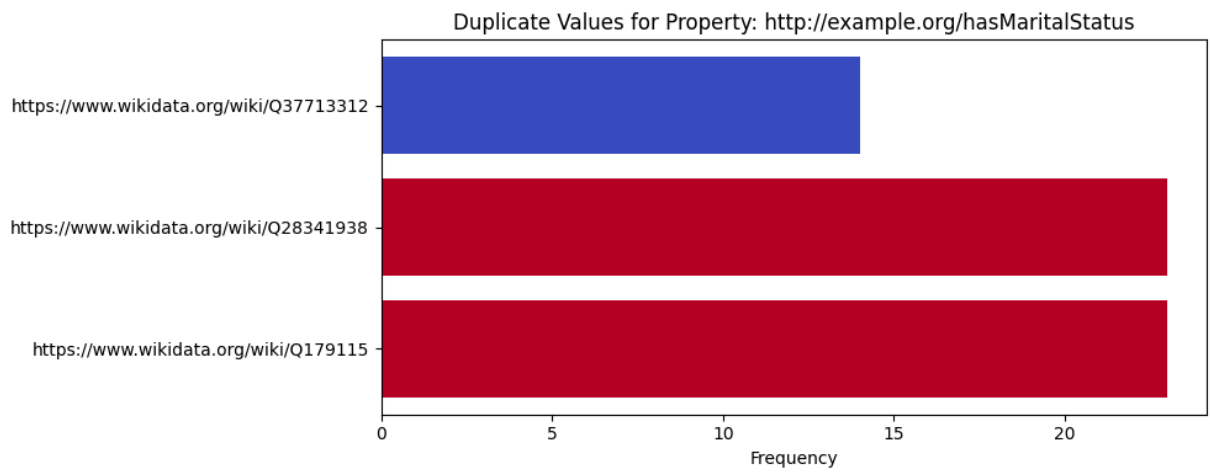
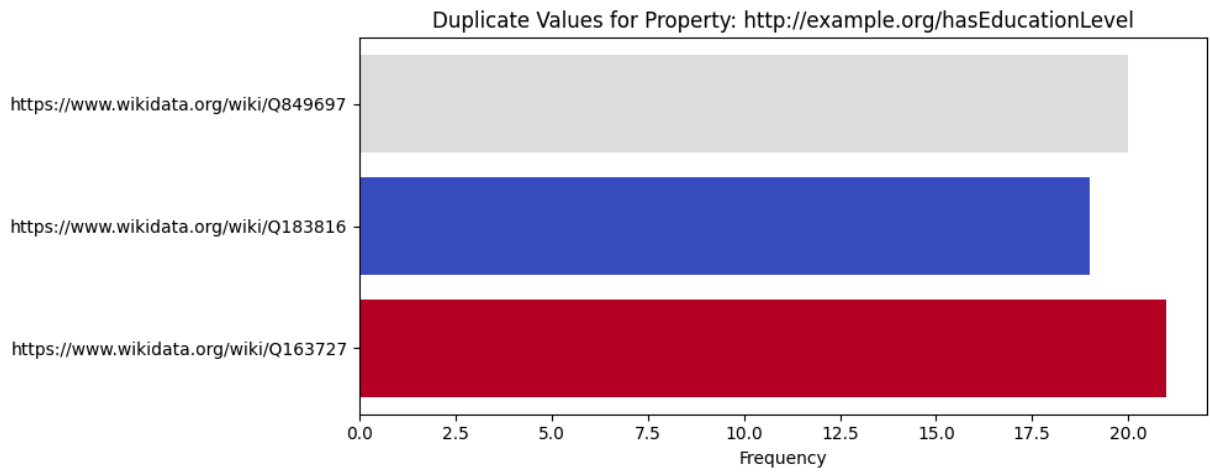
Duplicate Values for Property: <http://example.org/isAllergicTo>

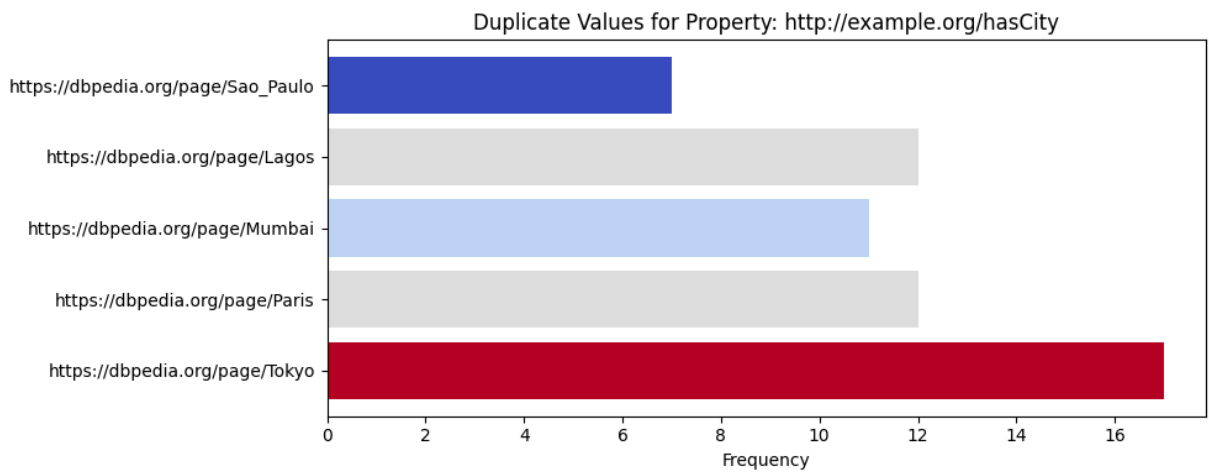
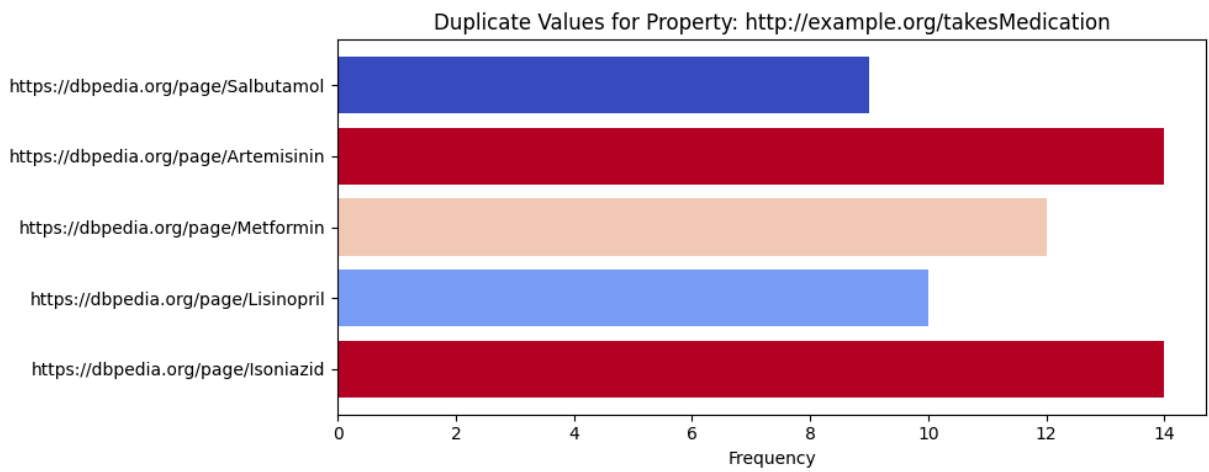
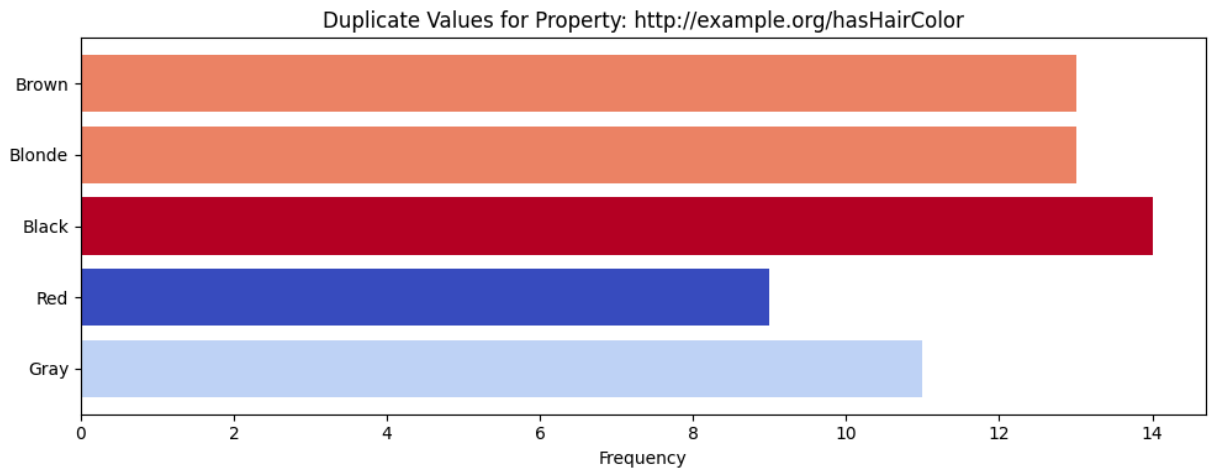


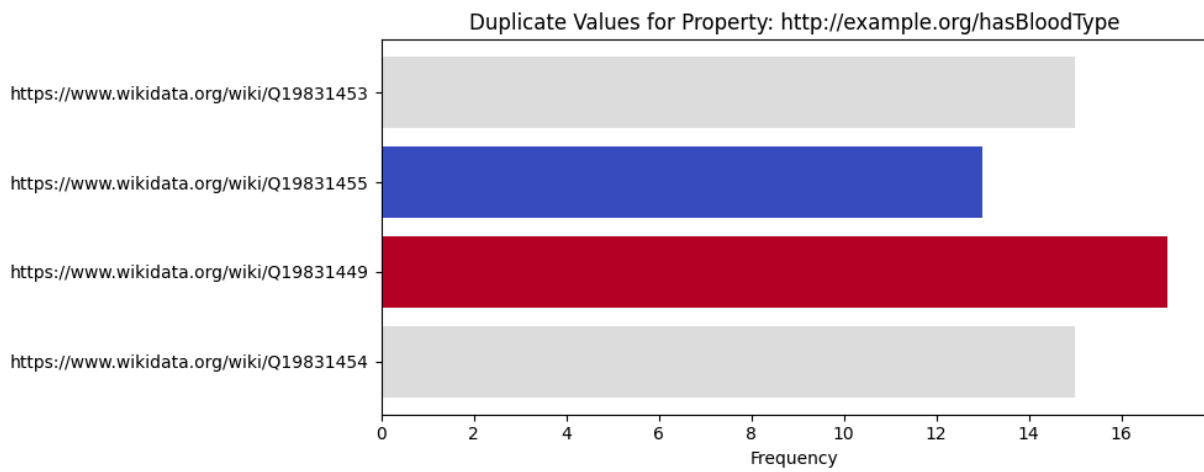
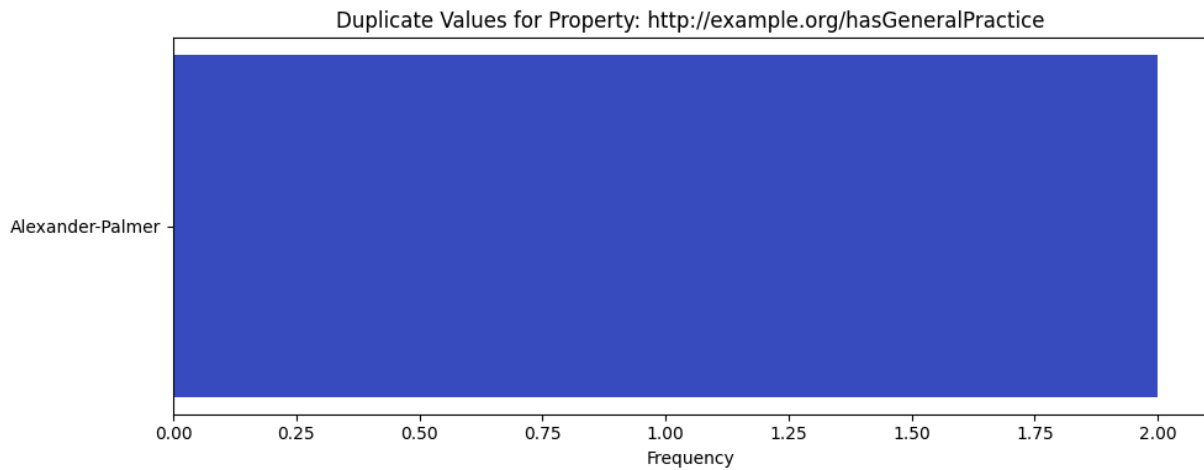
Duplicate Values for Property: <http://example.org/spokenIn>











```
In [9]: import os
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.cm as cm
import matplotlib.colors as mcolors
import seaborn as sns
import textwrap

# Set a clean style
plt.style.use("seaborn-v0_8-whitegrid")

# Ensure the folder exists
output_folder = "plots_image"
os.makedirs(output_folder, exist_ok=True)

# Helper: Wrap long labels
def wrap_labels(labels, width=30):
    return ['\n'.join(textwrap.wrap(label, width)) for label in labels]

for prop, dup_vals in duplicate_summary.items():
    plt.figure(figsize=(12, max(4, len(dup_vals) * 0.5)))

    labels = [str(v) for v in dup_vals.keys()]
    counts = list(dup_vals.values())

    # Sort by count
```



```

sorted_items = sorted(zip(labels, counts), key=lambda x: x[1])
labels, counts = zip(*sorted_items)

# Color gradient
norm = mcolors.Normalize(vmin=min(counts), vmax=max(counts))
cmap = cm.get_cmap("viridis")
colors = [cmap(norm(c)) for c in counts]

# Bar plot
bars = plt.barh(wrap_labels(labels), counts, color=colors)

# Add value labels
for bar, count in zip(bars, counts):
    plt.text(bar.get_width() + 0.5, bar.get_y() + bar.get_height() / 2,
             str(count), va='center', fontsize=10)

plt.xlabel("Frequency")
plt.title(f"Top Duplicate Values for Property: {prop}", fontsize=14, weight='bold')
plt.tight_layout()

# Save plot as JPG
filename = f"{output_folder}/{prop.replace('/', '_').replace(' ', '_')}.jpg"
plt.savefig(filename, format='jpg', dpi=300, bbox_inches='tight')
plt.close() # Close the figure to free memory

```

```

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap = cm.get_cmap("viridis")
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap = cm.get_cmap("viridis")
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap = cm.get_cmap("viridis")
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap = cm.get_cmap("viridis")
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap = cm.get_cmap("viridis")
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap = cm.get_cmap("viridis")
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap = cm.get_cmap("viridis")
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap = cm.get_cmap("viridis")
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap = cm.get_cmap("viridis")

```

```

py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
    cmap = cm.get_cmap("viridis")
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
    cmap = cm.get_cmap("viridis")
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/3060944986.
py:32: MatplotlibDeprecationWarning: The get_cmap function was deprecated in
Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]
`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
    cmap = cm.get_cmap("viridis")

```

```

In [13]: import os
import matplotlib.pyplot as plt
from scipy.stats import skew, normaltest
import pandas as pd

# Ensure the folder exists
output_folder = "plots_image"
os.makedirs(output_folder, exist_ok=True)

def analyze_numeric(col):
    values = pd.to_numeric(df[col], errors='coerce').dropna()
    print(f"Column: {col}")
    print(f"Skewness: {skew(values)}")
    k2, p = normaltest(values)
    print(f"Normality test p-value: {p}")
    print("Distribution: ", "Likely normal" if p > 0.05 else "Not normal (sk

    plt.figure(figsize=(8, 5))
    plt.hist(values, bins=10, edgecolor='black', color='skyblue')
    plt.title(f"Histogram of {col}", fontsize=14, weight='bold')
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.tight_layout()

    # Save plot as JPG
    filename = f"{output_folder}/{col.replace('/', '_').replace(' ', '_')}.jpg"
    plt.savefig(filename, format='jpg', dpi=300, bbox_inches='tight')
    plt.close() # Close the plot to free memory

```

```

In [21]: import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import skew, normaltest

# Make sure plots folder exists
os.makedirs("plots", exist_ok=True)

def analyze_numeric(col):
    values = pd.to_numeric(df[col], errors='coerce').dropna()

```

```

skewness = skew(values)
k2, p = normaltest(values)
normality = "Likely normal" if p > 0.05 else "Not normal (skewed)"

print(f"Column: {col}")
print(f"Skewness: {skewness:.3f}")
print(f"Normality test p-value: {p:.3e}")
print(f"Distribution: {normality}")

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Histogram with KDE and vertical lines for mean/median
sns.histplot(values, bins='auto', kde=True, ax=axes[0], color='skyblue',
axes[0].axvline(values.mean(), color='red', linestyle='--', label='Mean')
axes[0].axvline(values.median(), color='green', linestyle='-', label='Me
axes[0].set_title(f"Histogram + KDE of {col}")
axes[0].set_xlabel(col)
axes[0].set_ylabel("Frequency")
axes[0].legend()

# Boxplot
sns.boxplot(x=values, ax=axes[1], color='lightcoral')
axes[1].set_title(f"Boxplot of {col}")
axes[1].set_xlabel(col)

plt.tight_layout()
plt.savefig(f"plots/{col}_distribution.jpg", dpi=300) # Save as JPG with
plt.show()

def analyze_categorical(col, top_n=20):
    series = df[col].explode() if df[col].apply(lambda x: isinstance(x, list)
counts = series.value_counts()

    print(f"Column: {col}")
    print(counts)

    if len(counts) > top_n:
        counts = counts.head(top_n)
        title_extra = f" (Top {top_n} categories)"
    else:
        title_extra = ""

    plt.figure(figsize=(10, 6))
    ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
    plt.title(f"Distribution of {col}{title_extra}")
    plt.xlabel("Count")
    plt.ylabel(col)

    for i, v in enumerate(counts.values):
        ax.text(v + max(counts.values)*0.01, i, f"{v}", color='black', va='c

    plt.tight_layout()
    plt.savefig(f"plots/{col}_distribution.jpg", dpi=300)
    plt.show()

```

```
# Example usage (make sure df is defined)
numeric_cols = ['hasAge', 'hasIncome']
categorical_cols = ['hasBloodType', 'hasCountry', 'hasHairColor', 'spokenIn']

for col in numeric_cols:
    analyze_numeric(col)

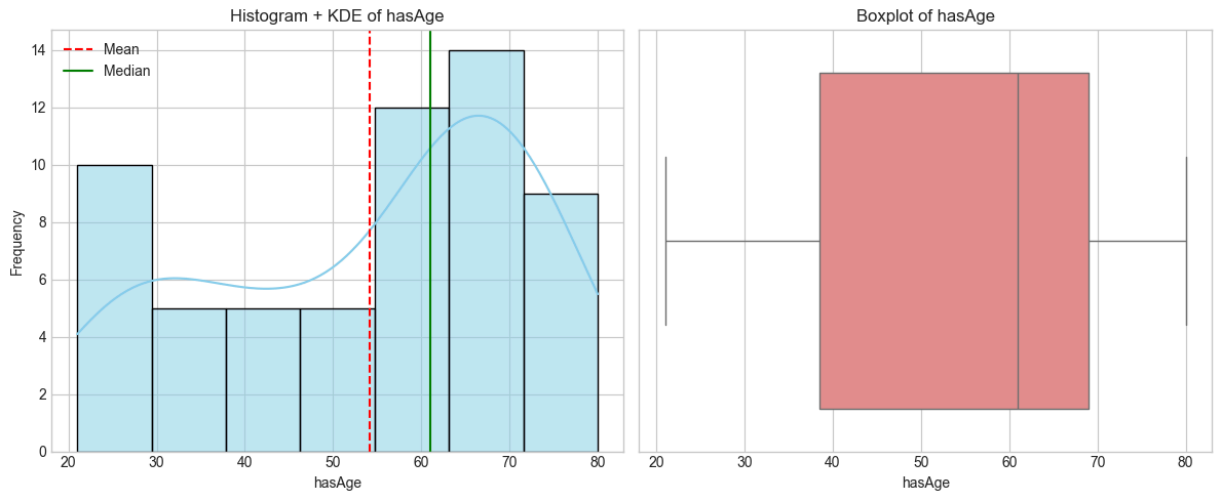
for col in categorical_cols:
    analyze_categorical(col)
```

Column: hasAge

Skewness: -0.474

Normality test p-value: $7.754e-04$

Distribution: Not normal (skewed)

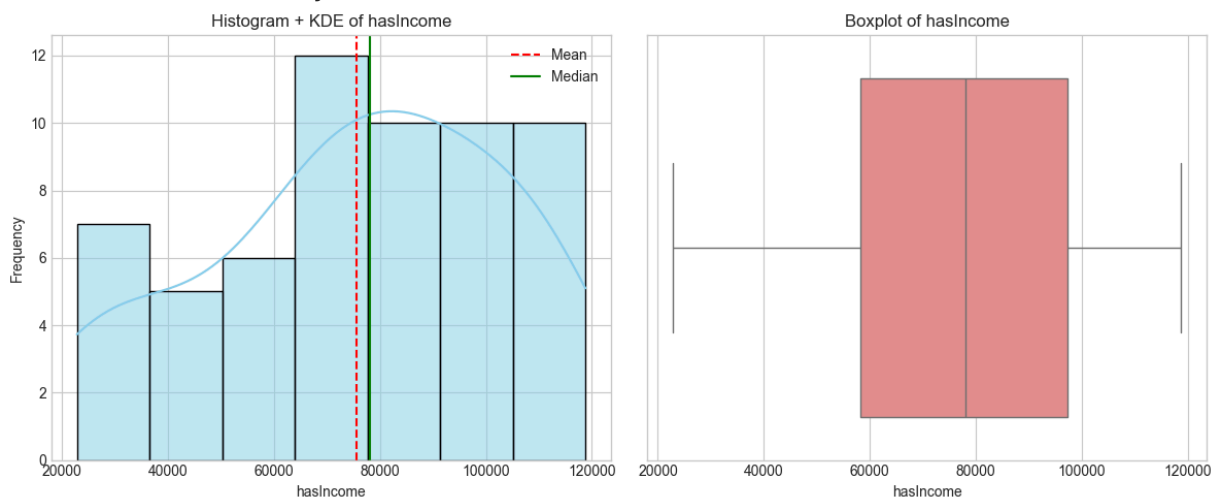


Column: hasIncome

Skewness: -0.349

Normality test p-value: $8.464e-02$

Distribution: Likely normal



Column: hasBloodType

hasBloodType

<https://www.wikidata.org/wiki/Q19831449> 17

<https://www.wikidata.org/wiki/Q19831454> 15

<https://www.wikidata.org/wiki/Q19831453> 15

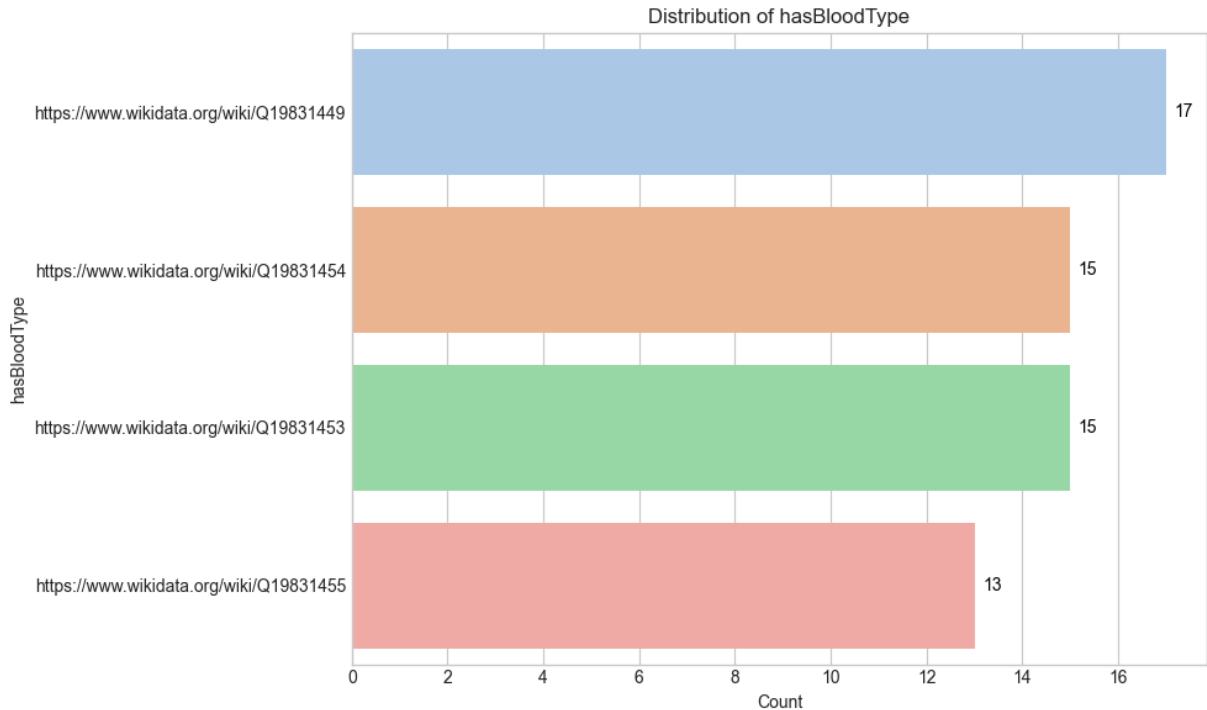
<https://www.wikidata.org/wiki/Q19831455> 13

Name: count, dtype: int64

```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/154815459.p  
y:56: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
```



Column: hasCountry

hasCountry

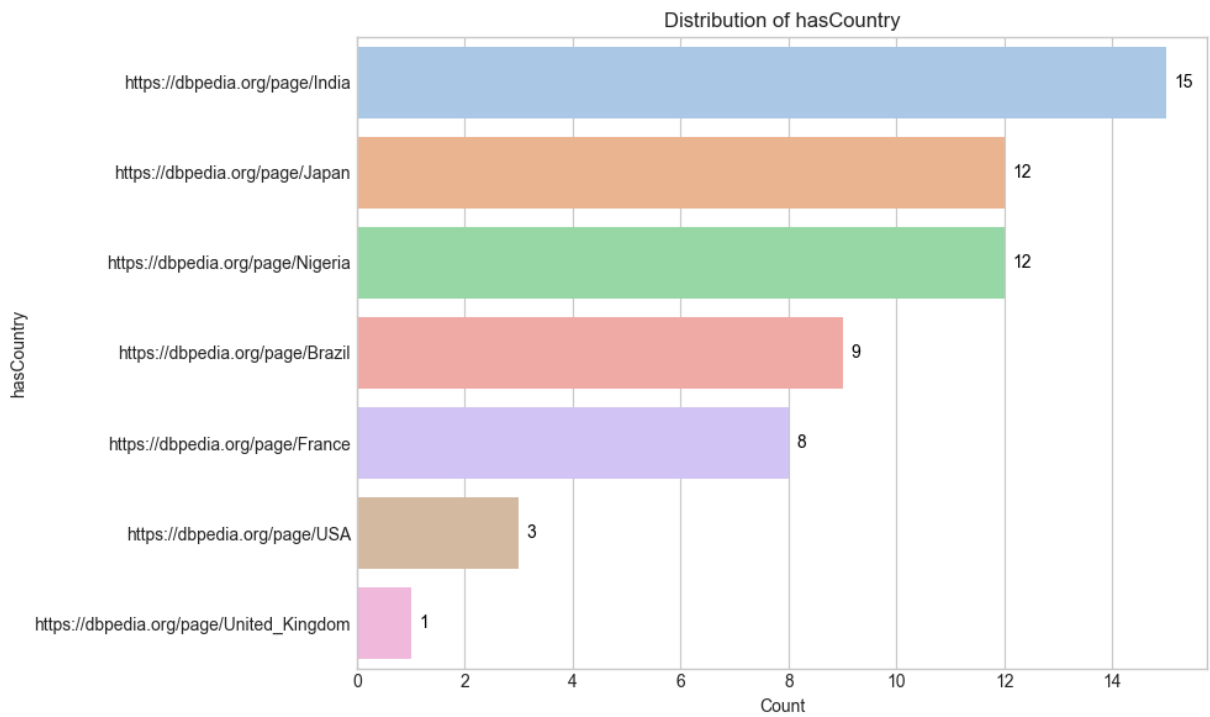
https://dbpedia.org/page/India	15
https://dbpedia.org/page/Japan	12
https://dbpedia.org/page/Nigeria	12
https://dbpedia.org/page/Brazil	9
https://dbpedia.org/page/France	8
https://dbpedia.org/page/USA	3
https://dbpedia.org/page/United_Kingdom	1

Name: count, dtype: int64

```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/154815459.p  
y:56: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
```



Column: hasHairColor

hasHairColor

Black 14

Brown 13

Blonde 13

Gray 11

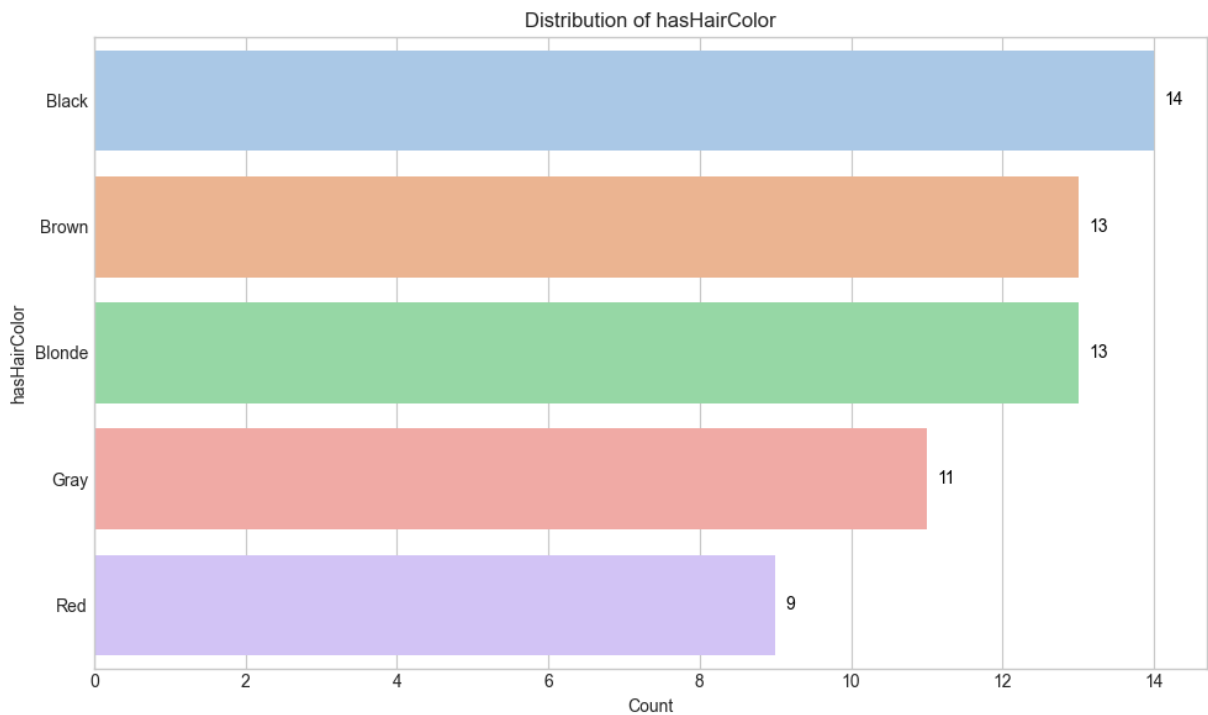
Red 9

Name: count, dtype: int64

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/154815459.py:56: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
```



Column: spokenIn

spokenIn

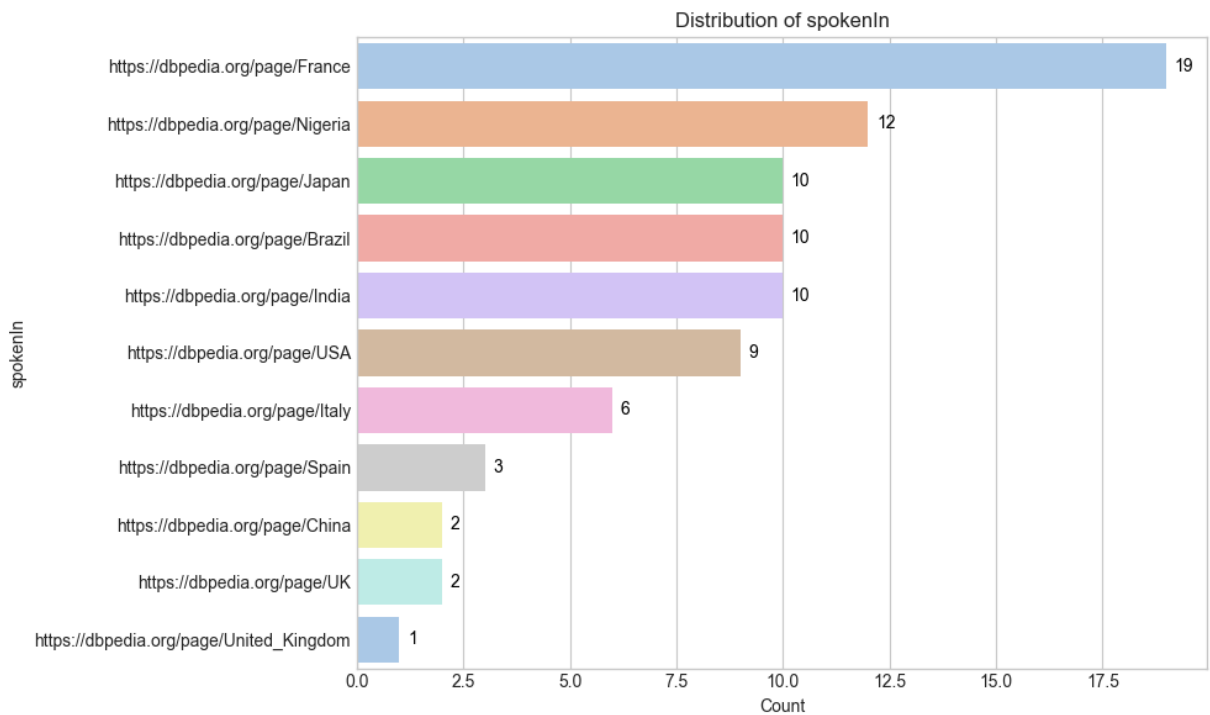
https://dbpedia.org/page/France	19
https://dbpedia.org/page/Nigeria	12
https://dbpedia.org/page/Japan	10
https://dbpedia.org/page/Brazil	10
https://dbpedia.org/page/India	10
https://dbpedia.org/page/USA	9
https://dbpedia.org/page/Italy	6
https://dbpedia.org/page/Spain	3
https://dbpedia.org/page/China	2
https://dbpedia.org/page/UK	2
https://dbpedia.org/page/United_Kingdom	1

Name: count, dtype: int64

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/154815459.py:56: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
```

Column: isAllergicTo

isAllergicTo

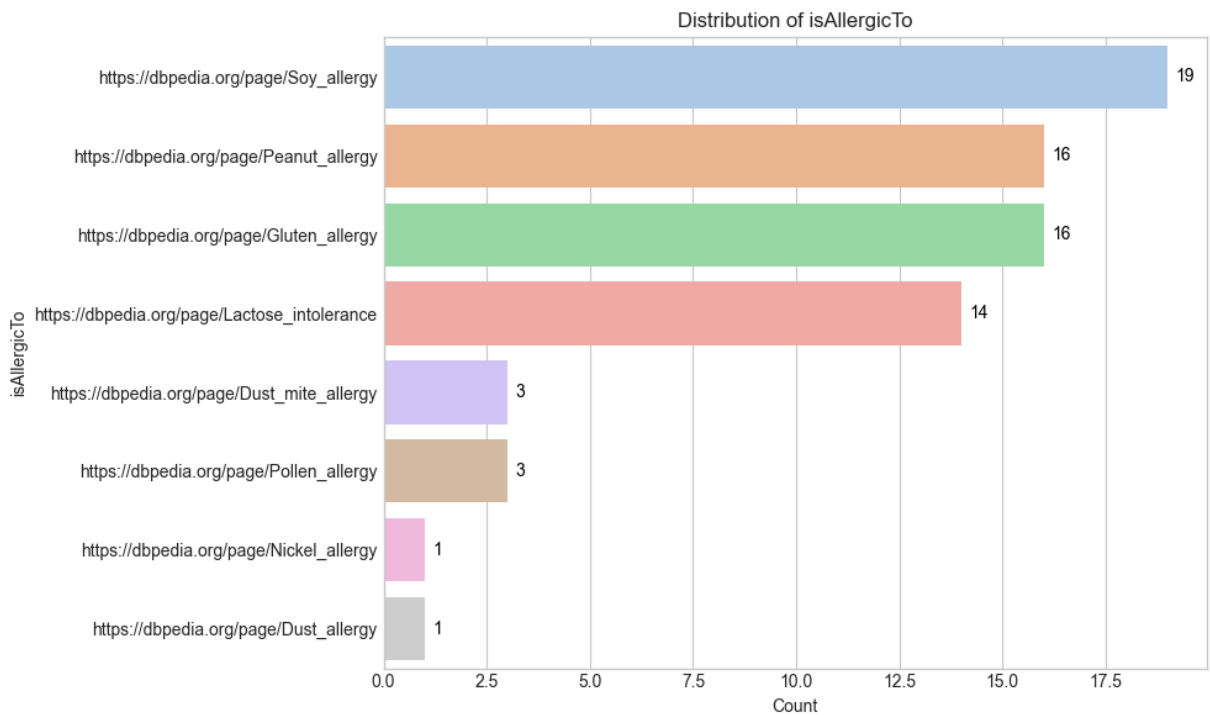
https://dbpedia.org/page/Soy_allergy	19
https://dbpedia.org/page/Peanut_allergy	16
https://dbpedia.org/page/Gluten_allergy	16
https://dbpedia.org/page/Lactose_intolerance	14
https://dbpedia.org/page/Dust_mite_allergy	3
https://dbpedia.org/page/Pollen_allergy	3
https://dbpedia.org/page/Nickel_allergy	1
https://dbpedia.org/page/Dust_allergy	1

Name: count, dtype: int64

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/154815459.py:56: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
```



```
In [20]: import matplotlib.pyplot as plt
import seaborn as sns

def analyze_categorical(col, top_n=20):
    # Flatten if column contains lists, otherwise keep as is
    series = df[col].explode() if df[col].apply(lambda x: isinstance(x, list)).sum() > 0 else df[col]
    counts = series.value_counts()

    print(f"Column: {col}")
    print(counts)

    # If too many categories, only show top_n for better plot readability
    if len(counts) > top_n:
        counts = counts.head(top_n)
        title_extra = f" (Top {top_n} categories)"
    else:
        title_extra = ""

    plt.figure(figsize=(10, 6))
    ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
    plt.title(f"Distribution of {col}{title_extra}")
    plt.xlabel("Count")
    plt.ylabel(col)

    # Add count labels on bars
    for i, v in enumerate(counts.values):
        ax.text(v + max(counts.values)*0.01, i, f"{v}", color='black', va='bottom')

    plt.tight_layout()
    plt.show()

categorical_cols = ['hasBloodType', 'hasCountry', 'hasHairColor', 'spokenIn']
```

```
for col in categorical_cols:
    analyze_categorical(col)
```

Column: hasBloodType

hasBloodType

<https://www.wikidata.org/wiki/Q19831449> 17

<https://www.wikidata.org/wiki/Q19831454> 15

<https://www.wikidata.org/wiki/Q19831453> 15

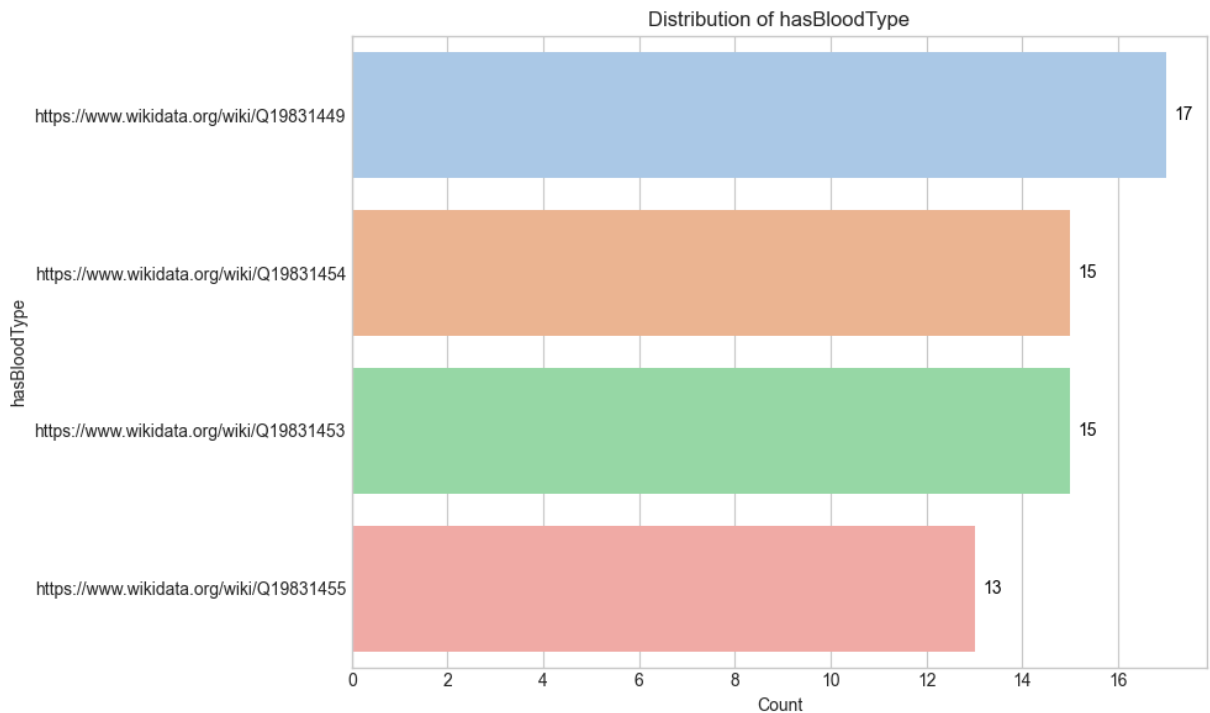
<https://www.wikidata.org/wiki/Q19831455> 13

Name: count, dtype: int64

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/1796710036.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
```



Column: hasCountry

hasCountry

<https://dbpedia.org/page/India> 15

<https://dbpedia.org/page/Japan> 12

<https://dbpedia.org/page/Nigeria> 12

<https://dbpedia.org/page/Brazil> 9

<https://dbpedia.org/page/France> 8

<https://dbpedia.org/page/USA> 3

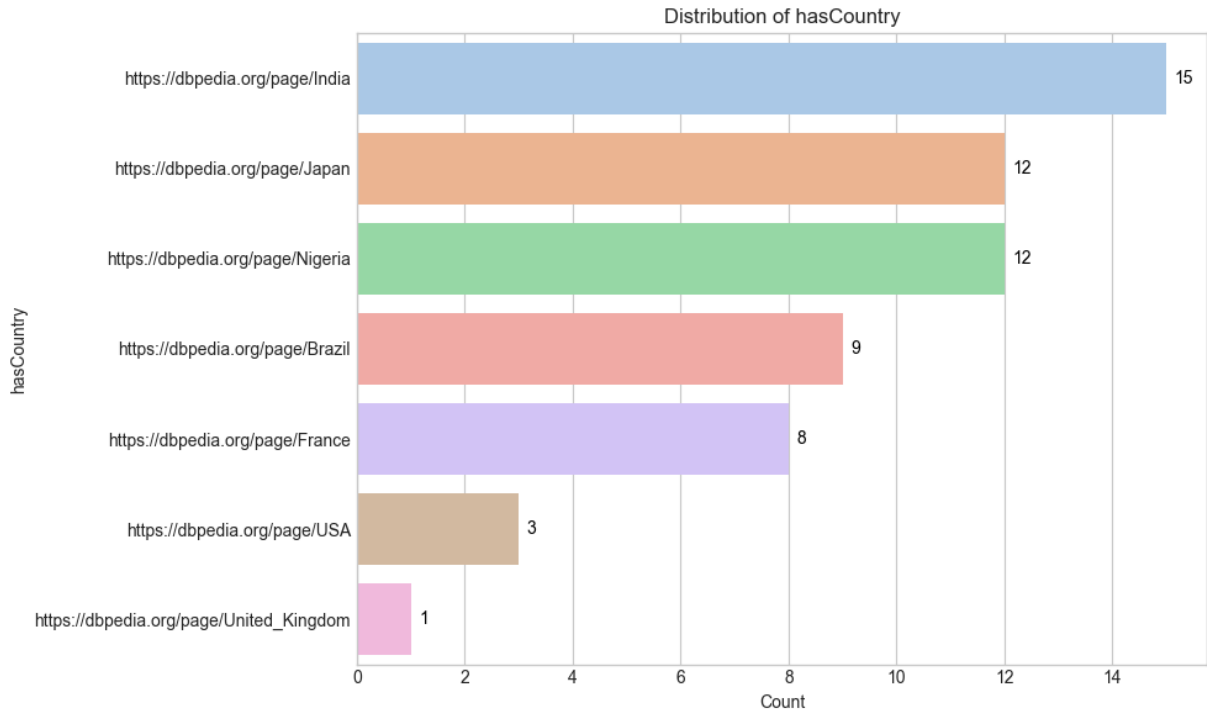
https://dbpedia.org/page/United_Kingdom 1

Name: count, dtype: int64

```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/1796710036.py:20: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
```



Column: hasHairColor

hasHairColor

Black 14

Brown 13

Blonde 13

Gray 11

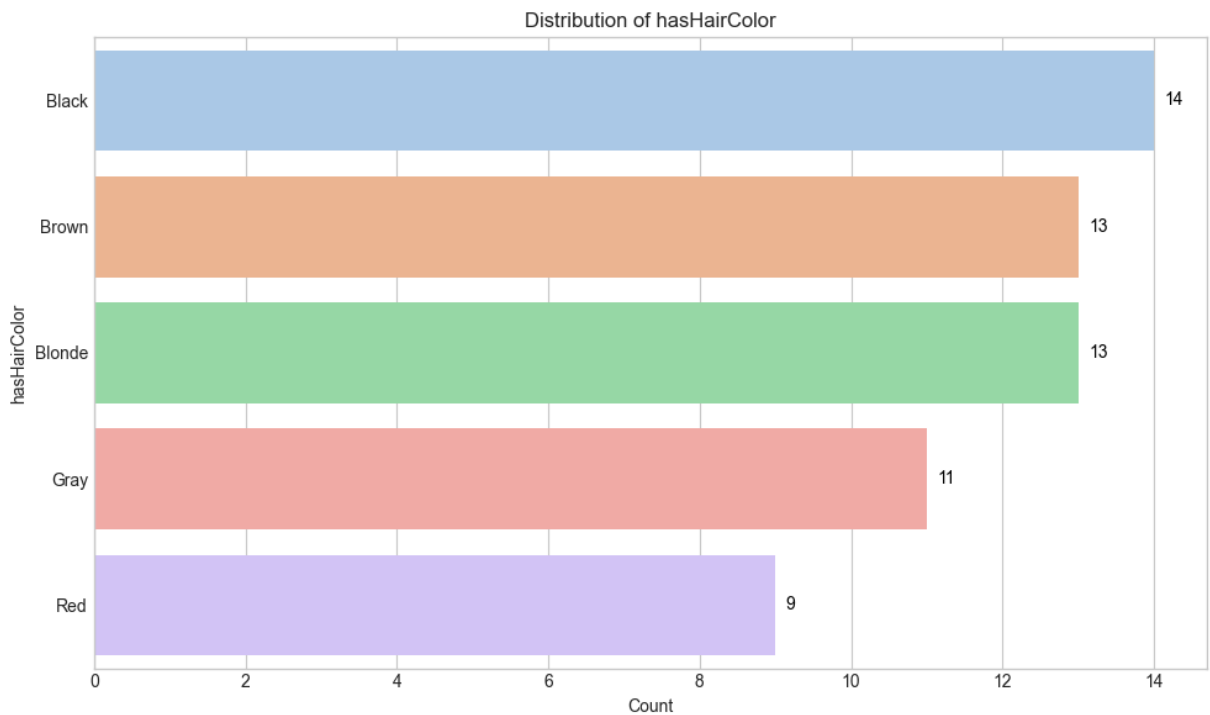
Red 9

Name: count, dtype: int64

```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/1796710036.py:20: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
```



Column: spokenIn

spokenIn

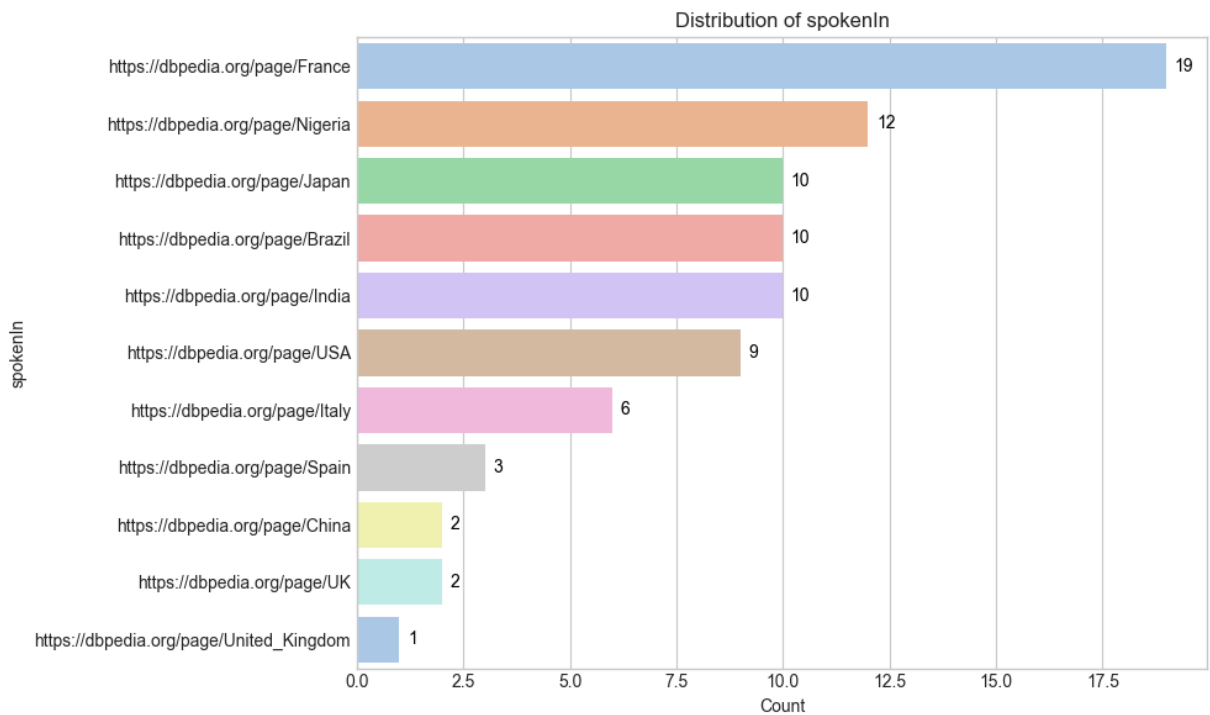
https://dbpedia.org/page/France	19
https://dbpedia.org/page/Nigeria	12
https://dbpedia.org/page/Japan	10
https://dbpedia.org/page/Brazil	10
https://dbpedia.org/page/India	10
https://dbpedia.org/page/USA	9
https://dbpedia.org/page/Italy	6
https://dbpedia.org/page/Spain	3
https://dbpedia.org/page/China	2
https://dbpedia.org/page/UK	2
https://dbpedia.org/page/United_Kingdom	1

Name: count, dtype: int64

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/1796710036.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
```



Column: isAllergicTo

isAllergicTo

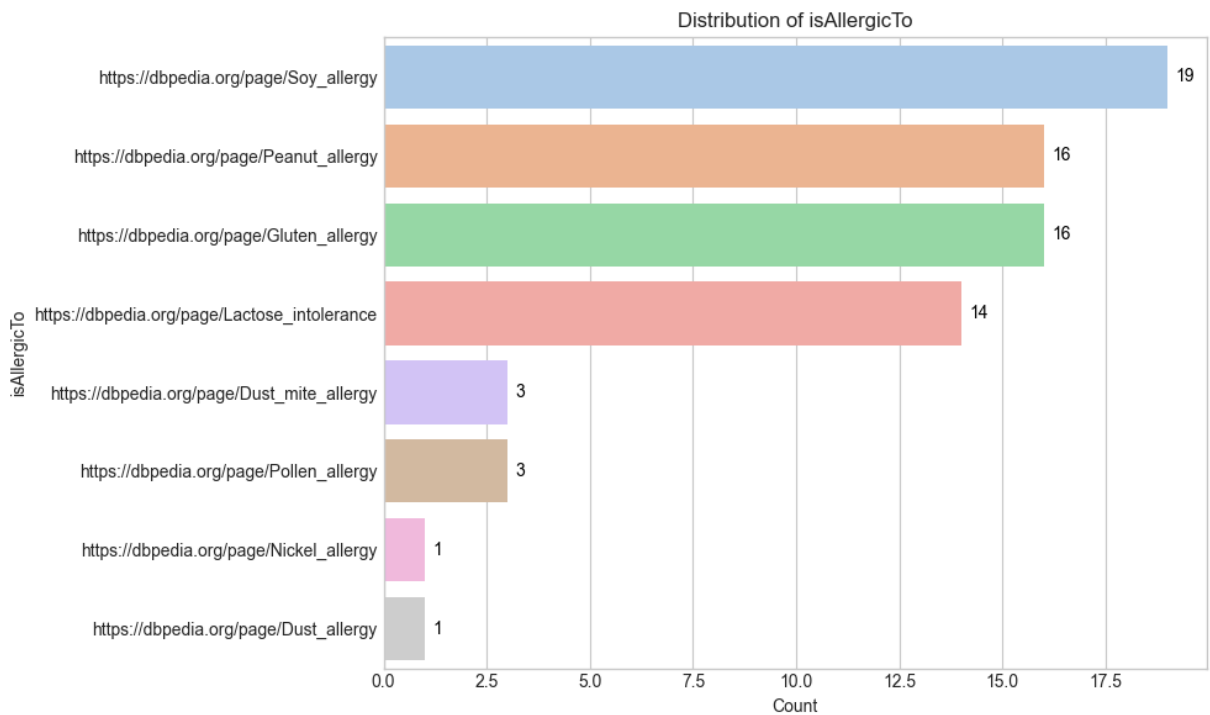
https://dbpedia.org/page/Soy_allergy	19
https://dbpedia.org/page/Peanut_allergy	16
https://dbpedia.org/page/Gluten_allergy	16
https://dbpedia.org/page/Lactose_intolerance	14
https://dbpedia.org/page/Dust_mite_allergy	3
https://dbpedia.org/page/Pollen_allergy	3
https://dbpedia.org/page/Nickel_allergy	1
https://dbpedia.org/page/Dust_allergy	1

Name: count, dtype: int64

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/1796710036.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.values, y=counts.index, palette='pastel')
```



```
In [14]: import matplotlib.pyplot as plt
from scipy.stats import skew, normaltest

def analyze_numeric(col):
    values = pd.to_numeric(df[col], errors='coerce').dropna()
    print(f"Column: {col}")
    print(f"Skewness: {skew(values)}")
    k2, p = normaltest(values)
    print(f"Normality test p-value: {p}")
    print("Distribution: ", "Likely normal" if p > 0.05 else "Not normal (sk

    plt.hist(values, bins=10, edgecolor='black')
    plt.title(f"Histogram of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.show()

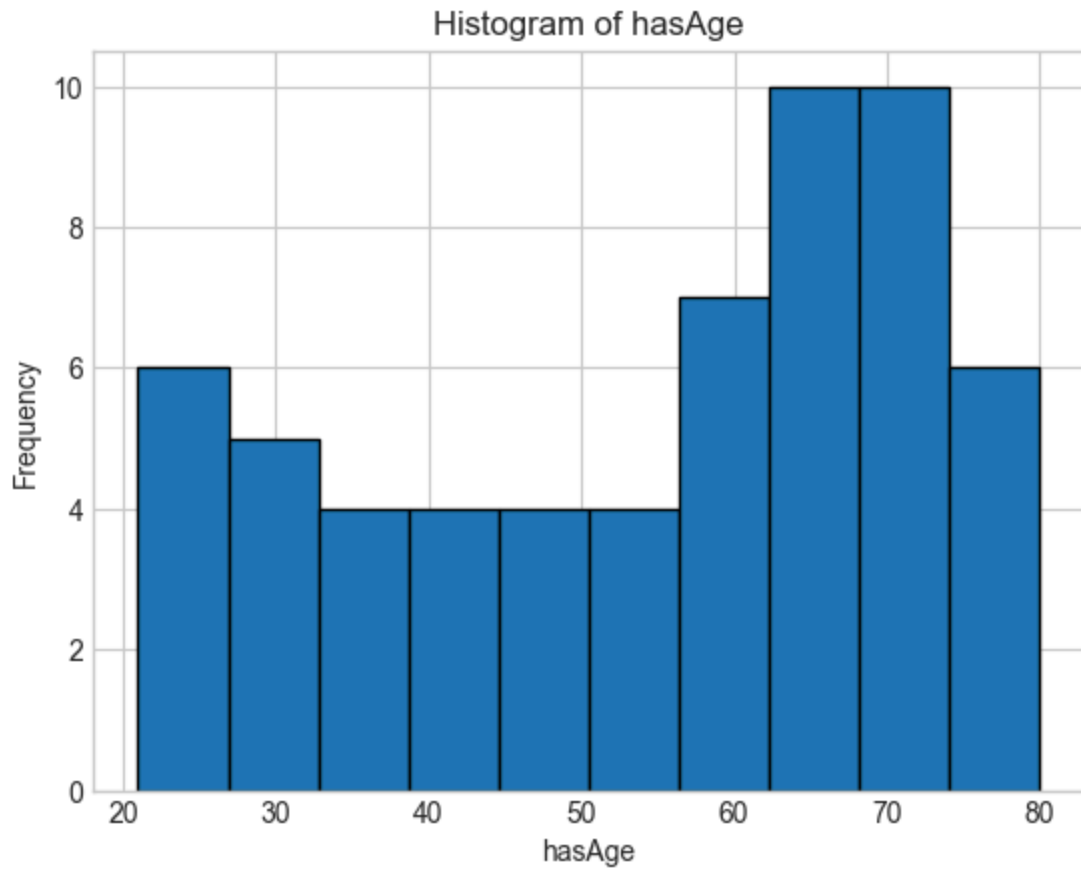
analyze_numeric('hasAge')
analyze_numeric('hasIncome')
```

Column: hasAge

Skewness: -0.47424780711740283

Normality test p-value: 0.0007753544736735845

Distribution: Not normal (skewed)

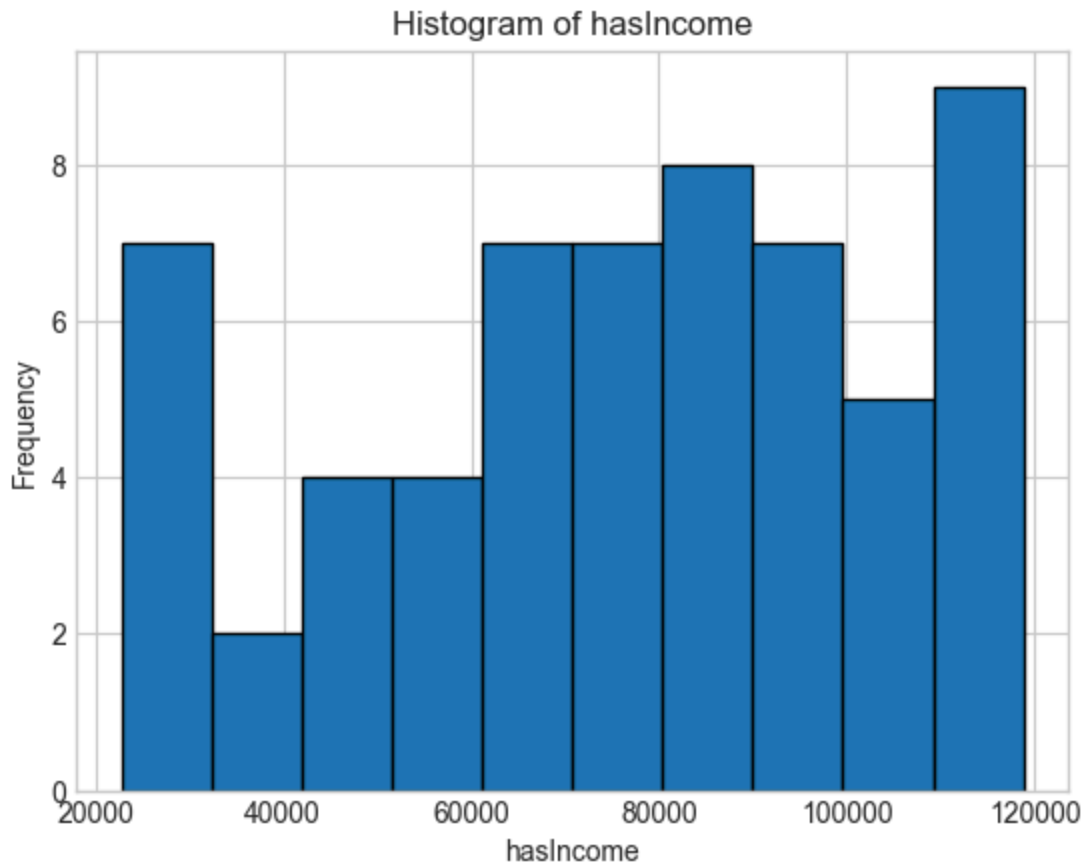


Column: hasIncome

Skewness: -0.34939076624857696

Normality test p-value: 0.08463696079678804

Distribution: Likely normal



```
In [19]: import matplotlib.pyplot as plt
import seaborn as sns

def analyze_categorical(col):
    series = df[col].explode() if df[col].apply(lambda x: isinstance(x, list)).sum() > 0 else df[col]
    counts = series.value_counts()

    print(f"Column: {col}")
    print(counts)

    plt.figure(figsize=(10,6))
    ax = sns.barplot(x=counts.index, y=counts.values, palette='viridis')

    plt.title(f"Distribution of {col}")
    plt.ylabel("Count")
    plt.xlabel(col)

    # Rotate x labels if too long or many
    plt.xticks(rotation=45, ha='right')

    # Add count labels on top of bars
    for i, count in enumerate(counts.values):
        ax.text(i, count + max(counts.values)*0.01, str(count), ha='center',
                color='red', fontweight='bold')

    plt.tight_layout()
    plt.savefig(f"{col}_distribution.jpg", dpi=300)
    plt.show()

categorical_cols = ['hasBloodType', 'hasCountry', 'hasHairColor', 'spokenIn']
```

```
for col in categorical_cols:
    analyze_categorical(col)
```

Column: hasBloodType

hasBloodType

<https://www.wikidata.org/wiki/Q19831449> 17

<https://www.wikidata.org/wiki/Q19831454> 15

<https://www.wikidata.org/wiki/Q19831453> 15

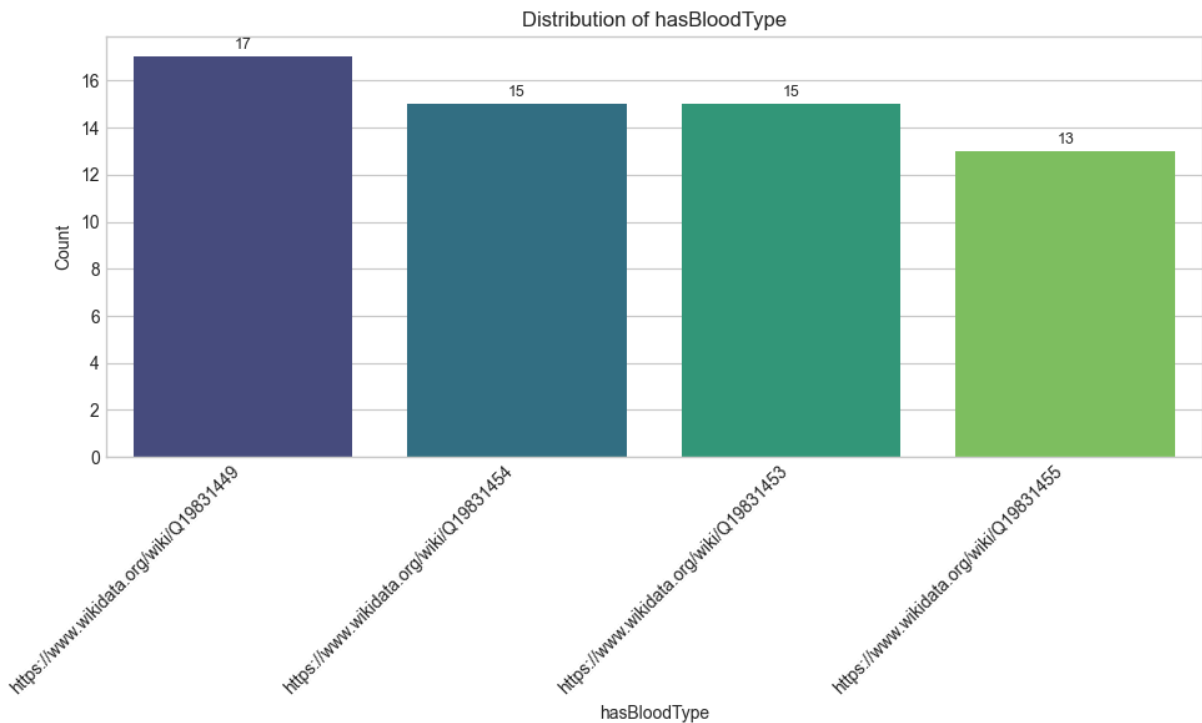
<https://www.wikidata.org/wiki/Q19831455> 13

Name: count, dtype: int64

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/4049963352.
py:12: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.index, y=counts.values, palette='viridis')
```



Column: hasCountry

hasCountry

<https://dbpedia.org/page/India> 15

<https://dbpedia.org/page/Japan> 12

<https://dbpedia.org/page/Nigeria> 12

<https://dbpedia.org/page/Brazil> 9

<https://dbpedia.org/page/France> 8

<https://dbpedia.org/page/USA> 3

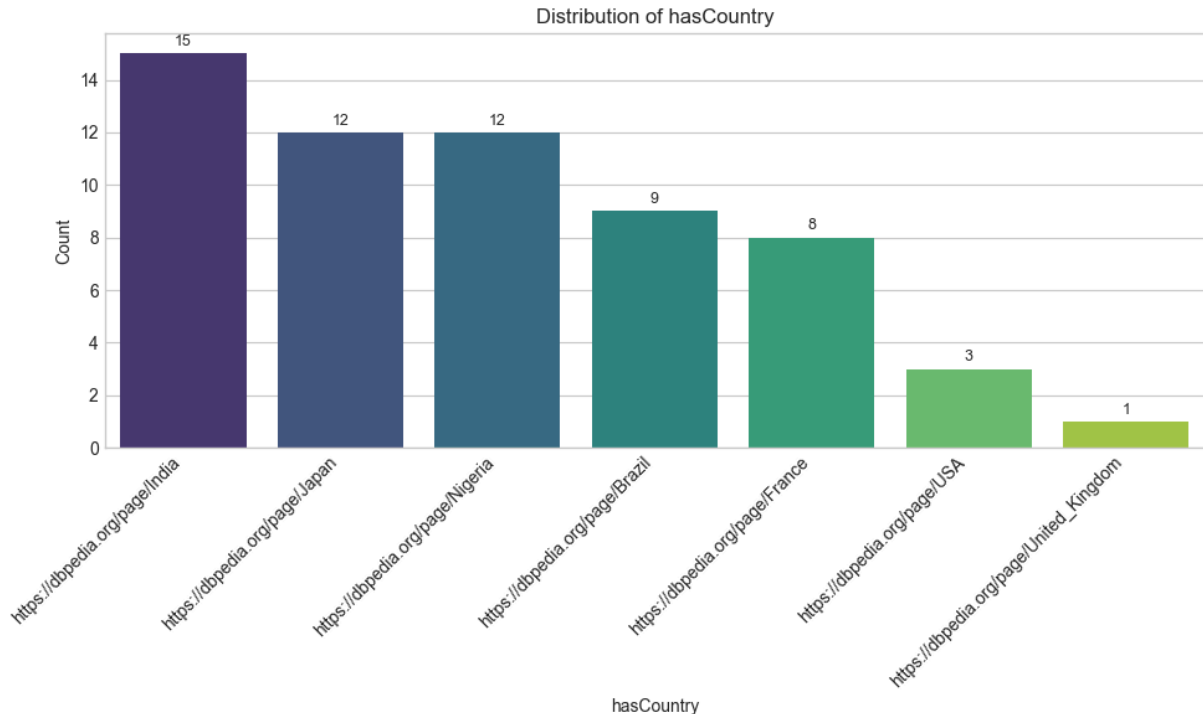
https://dbpedia.org/page/United_Kingdom 1

Name: count, dtype: int64

```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/4049963352.py:12: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.index, y=counts.values, palette='viridis')
```



Column: hasHairColor

hasHairColor

Black 14

Brown 13

Blonde 13

Gray 11

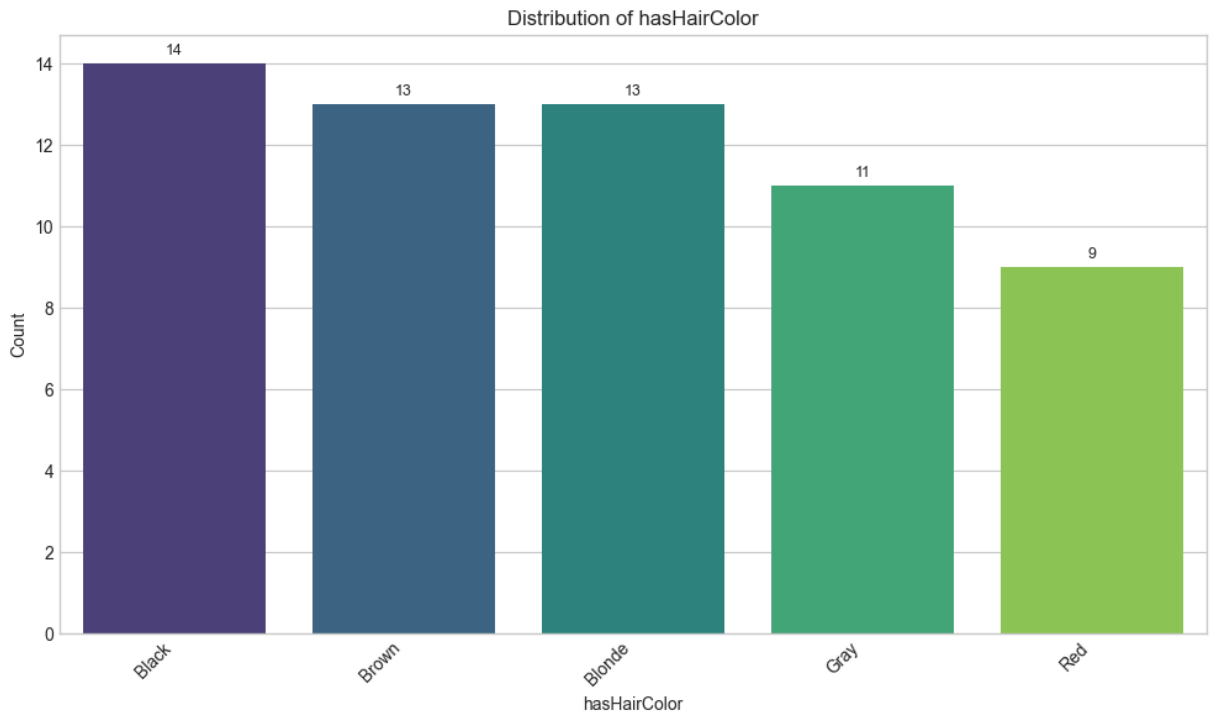
Red 9

Name: count, dtype: int64

```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/4049963352.py:12: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.index, y=counts.values, palette='viridis')
```



Column: spokenIn

spokenIn

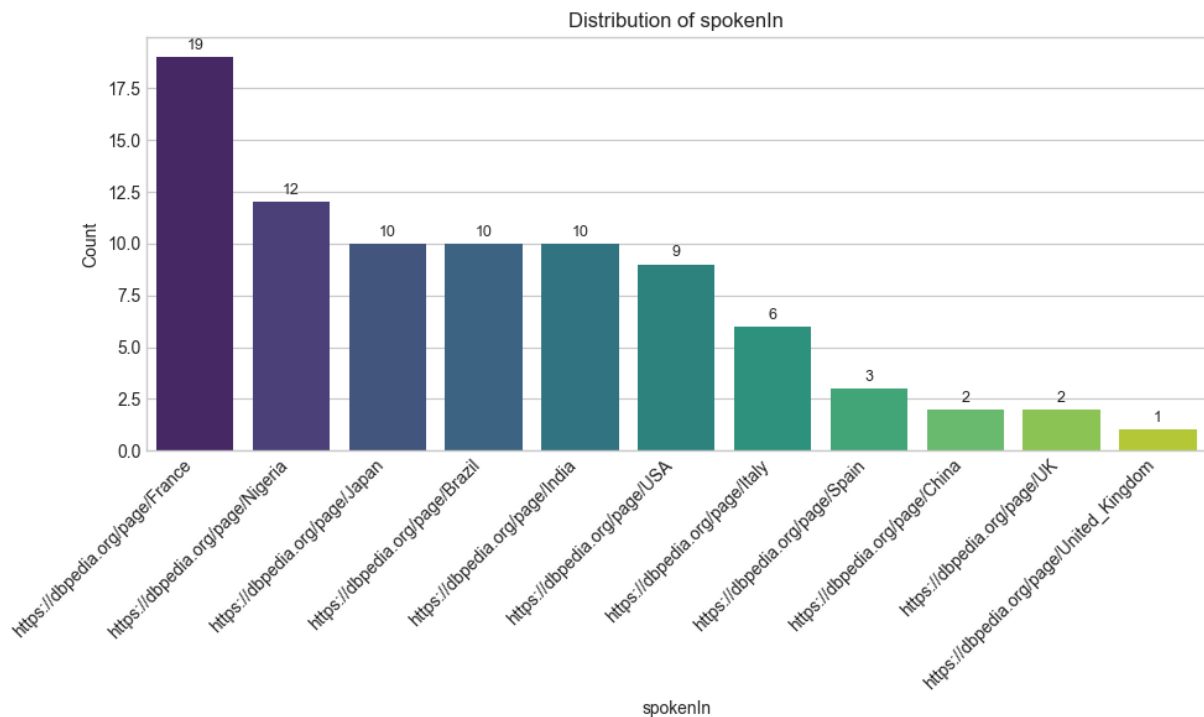
https://dbpedia.org/page/France	19
https://dbpedia.org/page/Nigeria	12
https://dbpedia.org/page/Japan	10
https://dbpedia.org/page/Brazil	10
https://dbpedia.org/page/India	10
https://dbpedia.org/page/USA	9
https://dbpedia.org/page/Italy	6
https://dbpedia.org/page/Spain	3
https://dbpedia.org/page/China	2
https://dbpedia.org/page/UK	2
https://dbpedia.org/page/United_Kingdom	1

Name: count, dtype: int64

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/4049963352.py:12: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.index, y=counts.values, palette='viridis')
```



Column: isAllergicTo

isAllergicTo

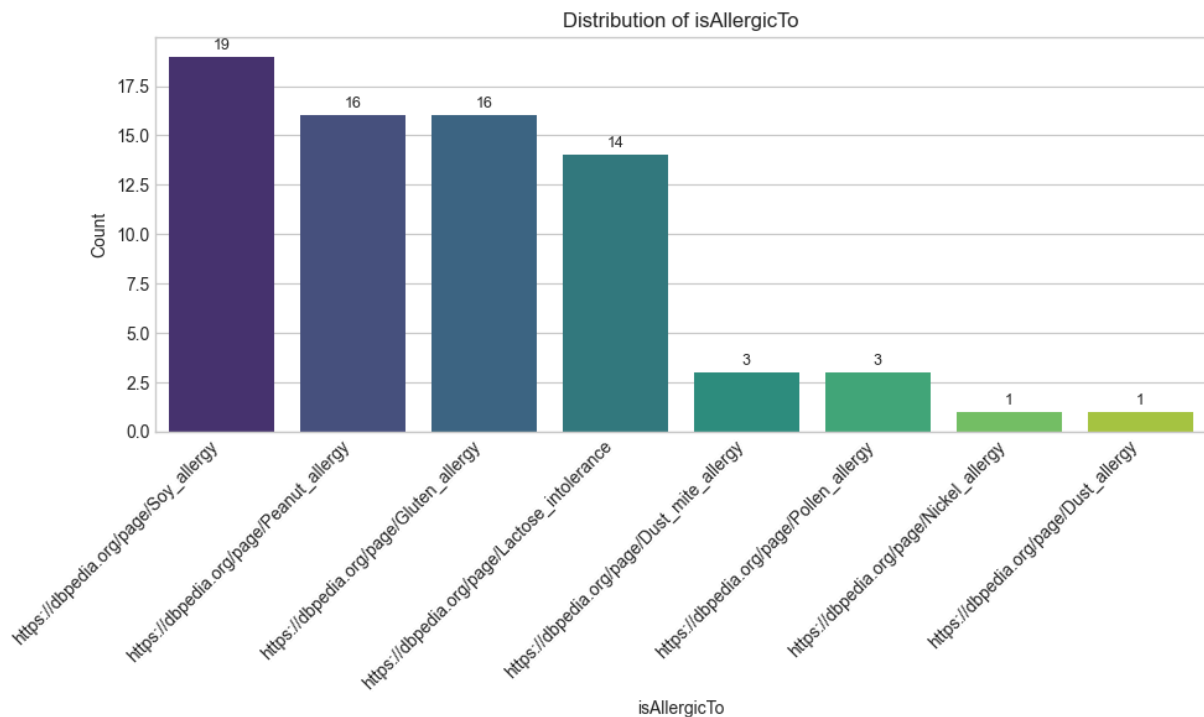
https://dbpedia.org/page/Soy_allergy	19
https://dbpedia.org/page/Peanut_allergy	16
https://dbpedia.org/page/Gluten_allergy	16
https://dbpedia.org/page/Lactose_intolerance	14
https://dbpedia.org/page/Dust_mite_allergy	3
https://dbpedia.org/page/Pollen_allergy	3
https://dbpedia.org/page/Nickel_allergy	1
https://dbpedia.org/page/Dust_allergy	1

Name: count, dtype: int64

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_29726/4049963352.py:12: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=counts.index, y=counts.values, palette='viridis')
```



```
In [8]: def analyze_categorical(col):
        series = df[col].explode() if df[col].apply(lambda x: isinstance(x, list))
        counts = series.value_counts()
        print(f"Column: {col}")
        print(counts)
        counts.plot(kind='bar', title=f"Distribution of {col}")
        plt.ylabel("Count")
        plt.show()

        categorical_cols = ['hasBloodType', 'hasCountry', 'hasHairColor', 'spokenIn']
        for col in categorical_cols:
            analyze_categorical(col)
```

Column: hasBloodType

hasBloodType

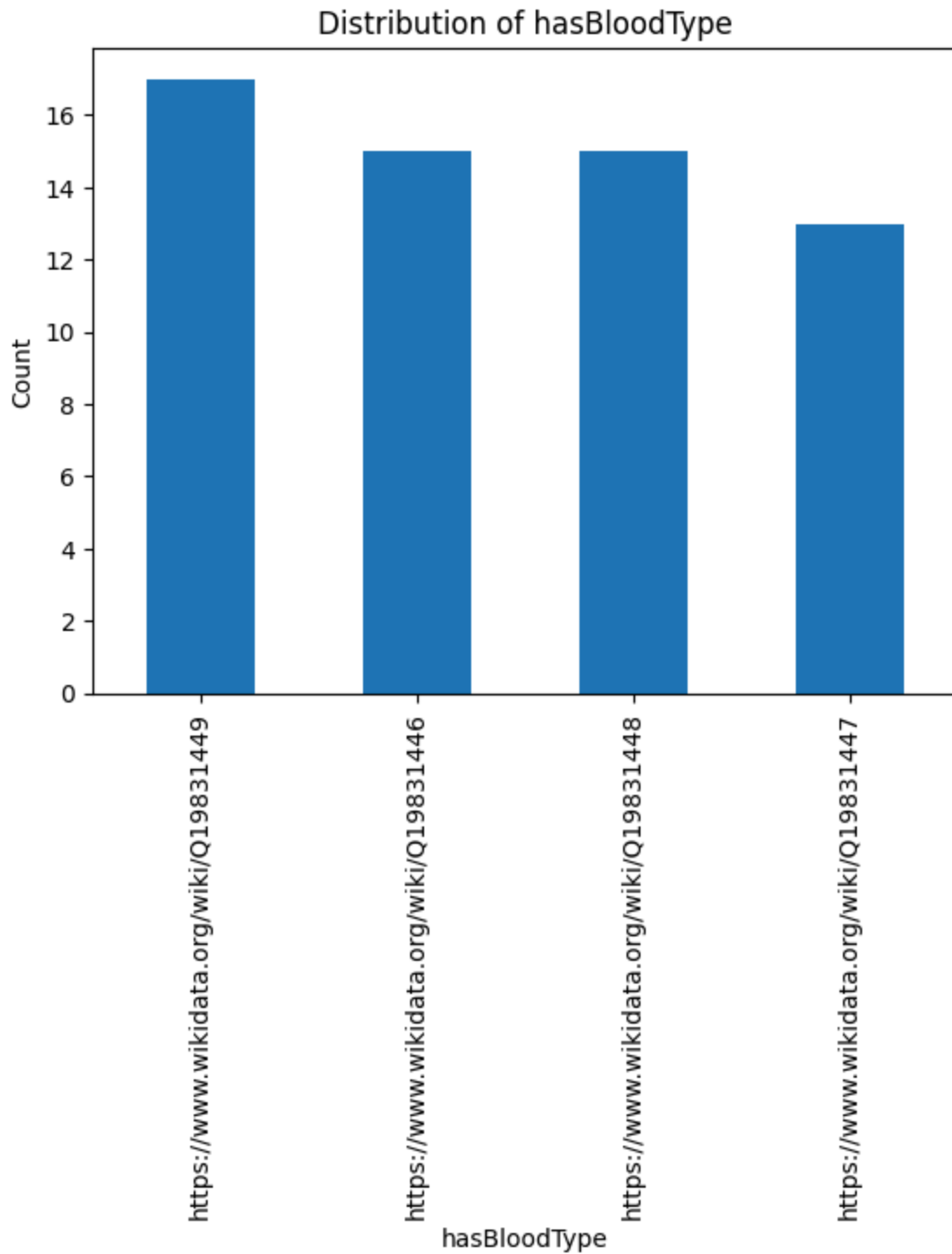
<https://www.wikidata.org/wiki/Q19831449> 17

<https://www.wikidata.org/wiki/Q19831446> 15

<https://www.wikidata.org/wiki/Q19831448> 15

<https://www.wikidata.org/wiki/Q19831447> 13

Name: count, dtype: int64

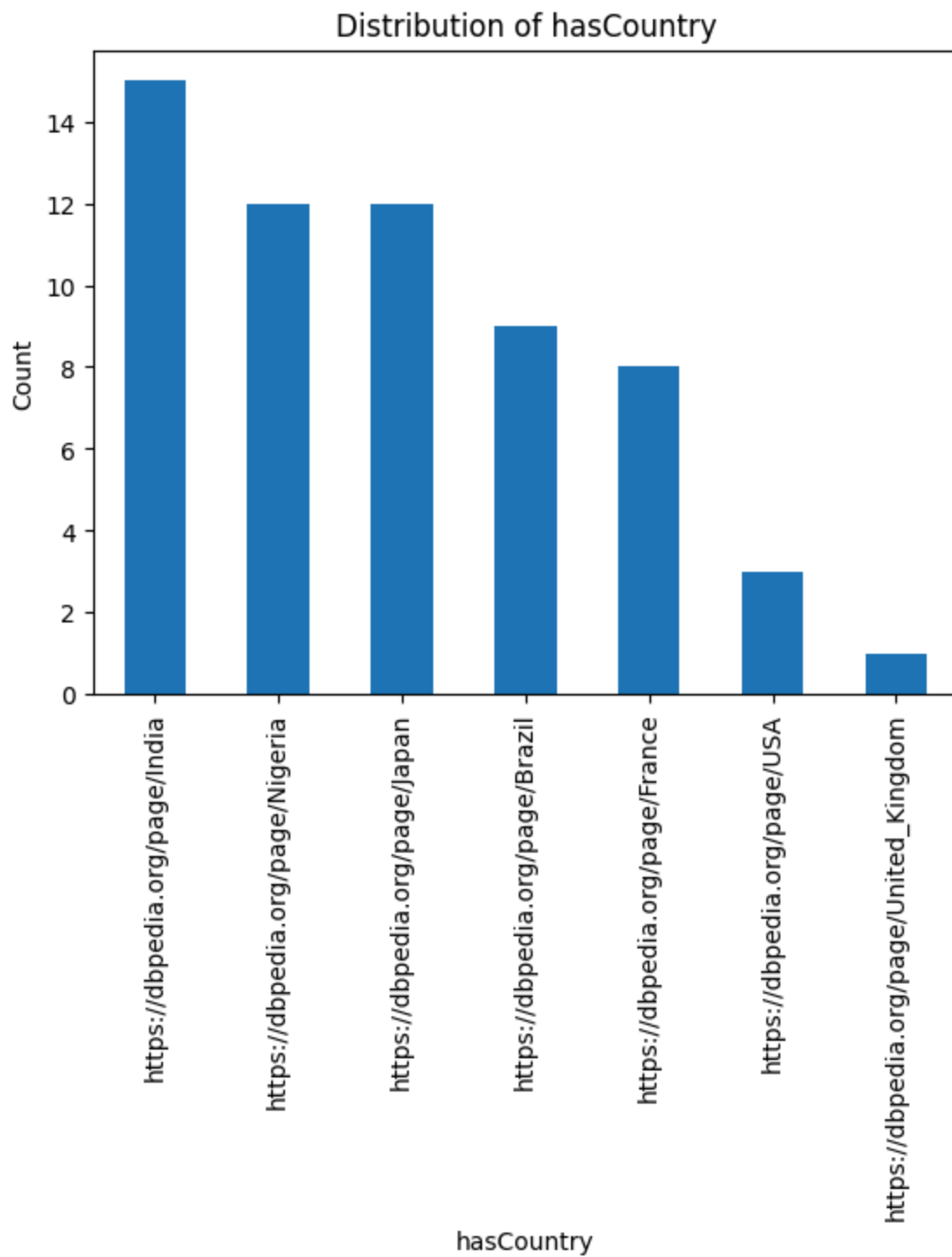


Column: hasCountry

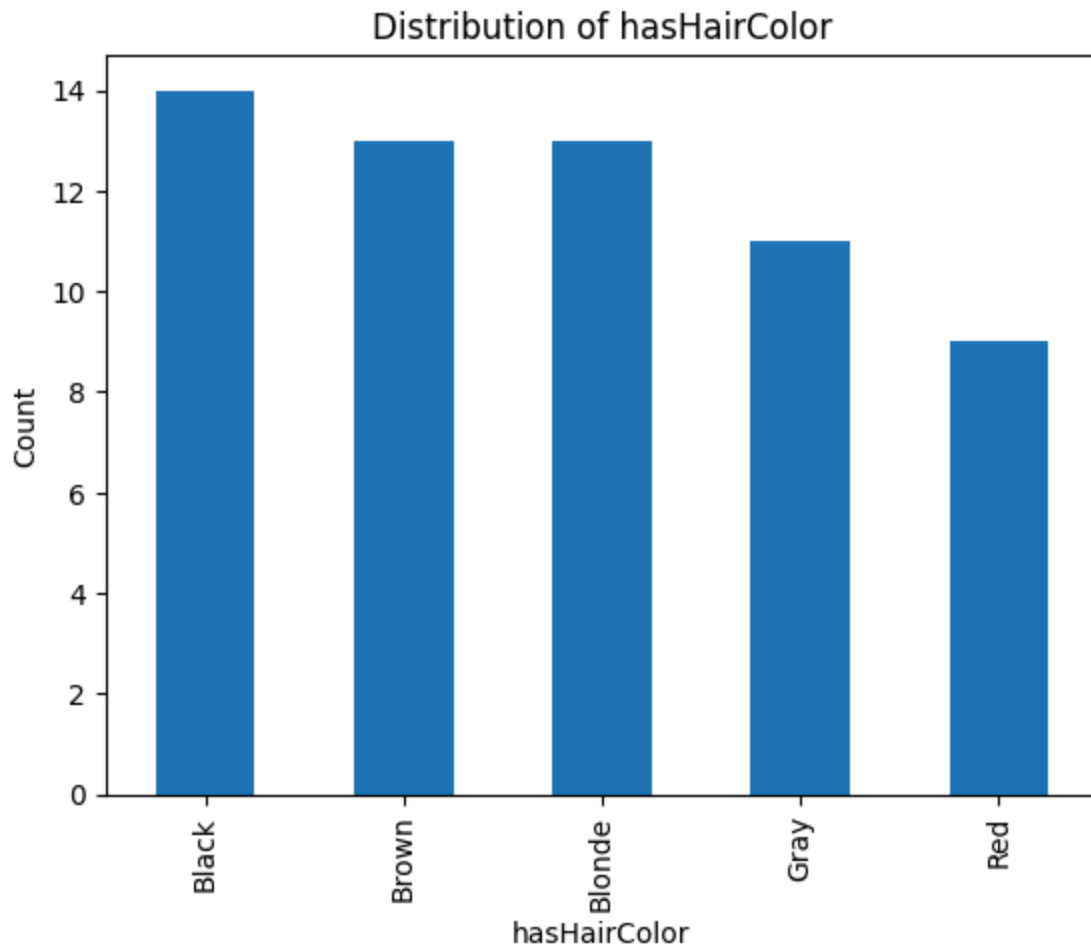
hasCountry

https://dbpedia.org/page/India	15
https://dbpedia.org/page/Nigeria	12
https://dbpedia.org/page/Japan	12
https://dbpedia.org/page/Brazil	9
https://dbpedia.org/page/France	8
https://dbpedia.org/page/USA	3
https://dbpedia.org/page/United_Kingdom	1

Name: count, dtype: int64



Column: hasHairColor
hasHairColor
Black 14
Brown 13
Blonde 13
Gray 11
Red 9
Name: count, dtype: int64

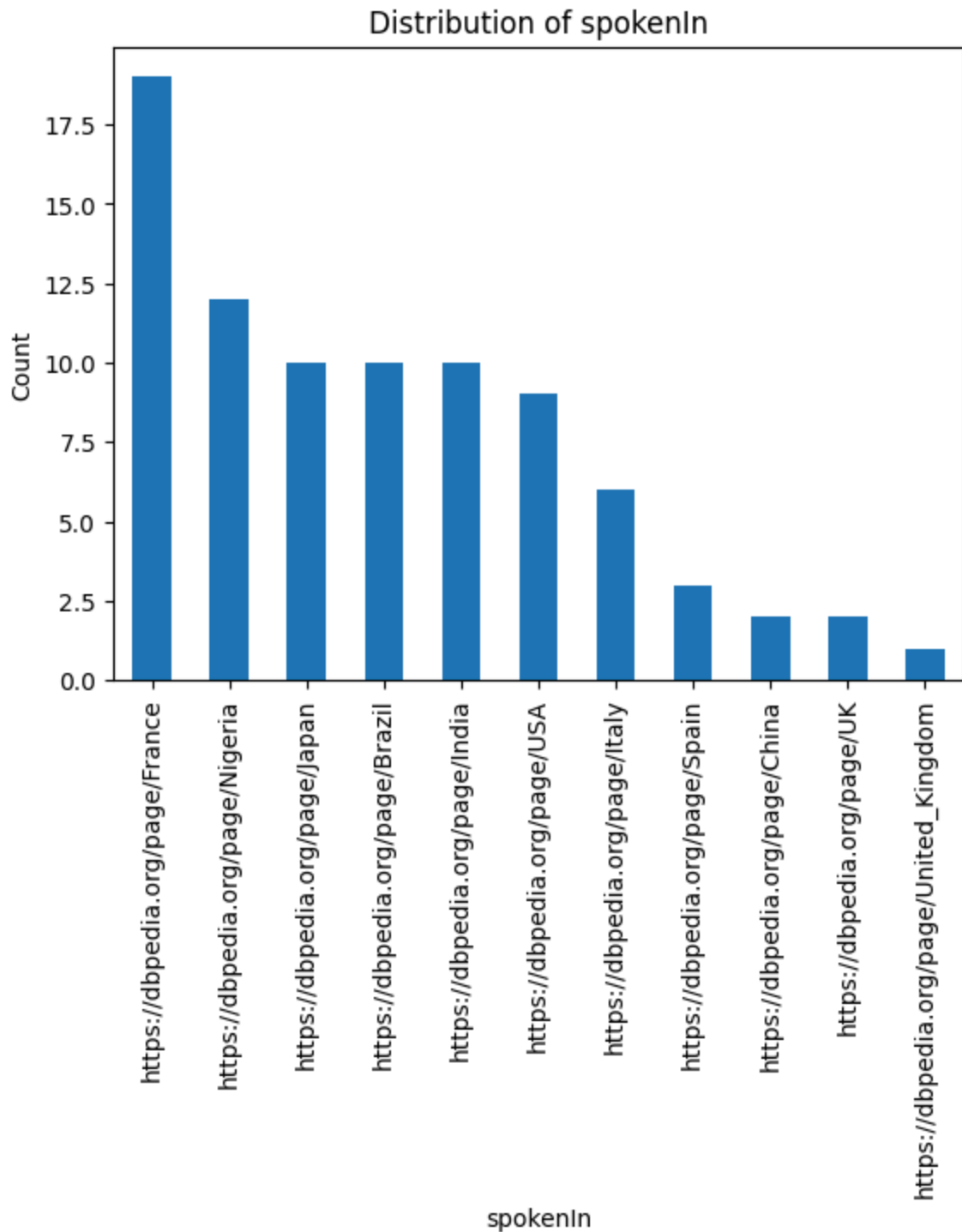


Column: spokenIn

spokenIn

https://dbpedia.org/page/France	19
https://dbpedia.org/page/Nigeria	12
https://dbpedia.org/page/Japan	10
https://dbpedia.org/page/Brazil	10
https://dbpedia.org/page/India	10
https://dbpedia.org/page/USA	9
https://dbpedia.org/page/Italy	6
https://dbpedia.org/page/Spain	3
https://dbpedia.org/page/China	2
https://dbpedia.org/page/UK	2
https://dbpedia.org/page/United_Kingdom	1

Name: count, dtype: int64

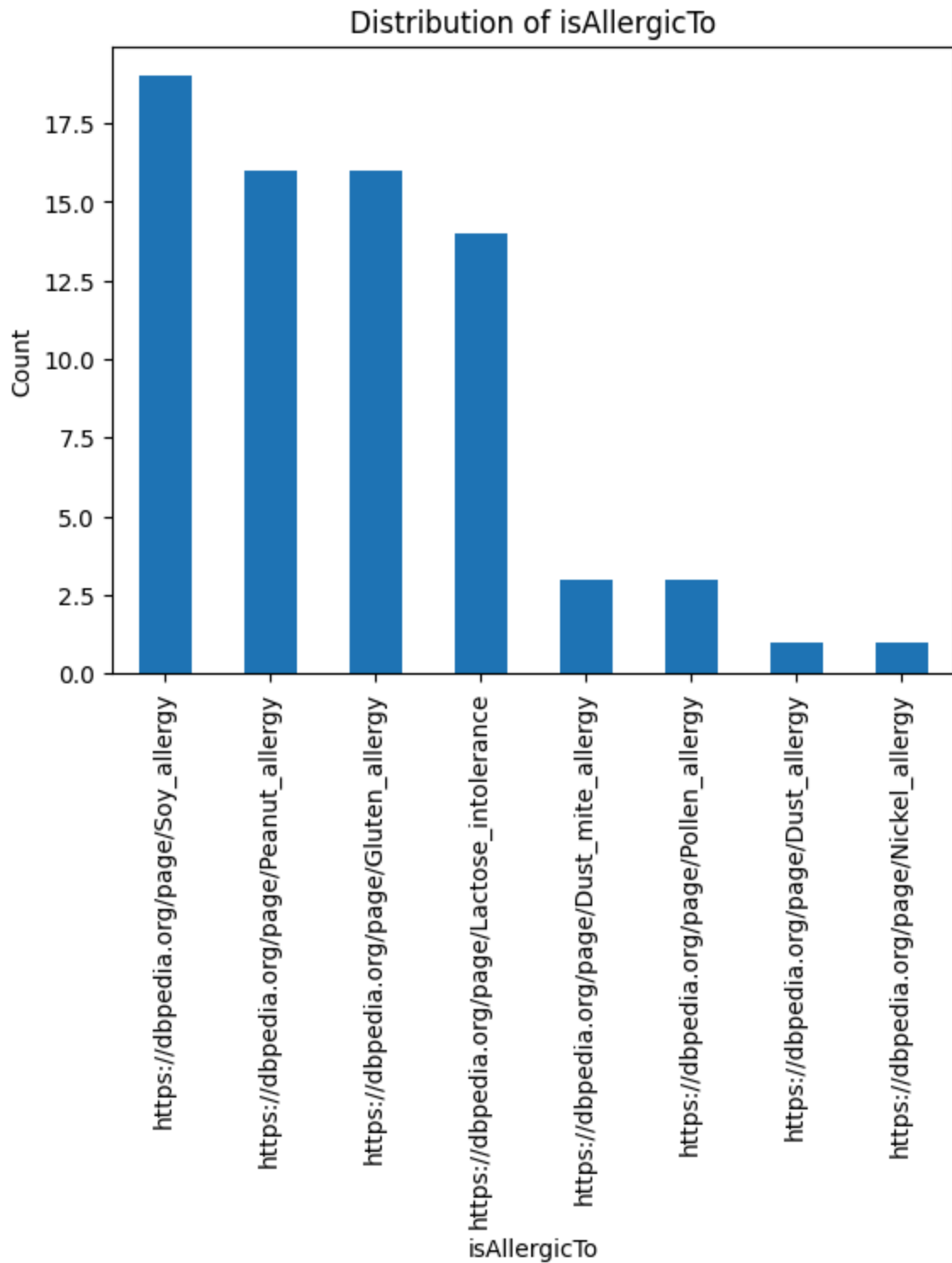


Column: isAllergicTo

isAllergicTo

https://dbpedia.org/page/Soy_allergy	19
https://dbpedia.org/page/Peanut_allergy	16
https://dbpedia.org/page/Gluten_allergy	16
https://dbpedia.org/page/Lactose_intolerance	14
https://dbpedia.org/page/Dust_mite_allergy	3
https://dbpedia.org/page/Pollen_allergy	3
https://dbpedia.org/page/Dust_allergy	1
https://dbpedia.org/page/Nickel_allergy	1

Name: count, dtype: int64



```
In [16]: import os
import matplotlib.pyplot as plt

# Ensure the folder exists
output_folder = "plots_image"
os.makedirs(output_folder, exist_ok=True)

def analyze_categorical(col):
    series = df[col].explode() if df[col].apply(lambda x: isinstance(x, list))
    counts = series.value_counts()
```

```

print(f"Column: {col}")
print(counts)

plt.figure(figsize=(10, 5))
counts.plot(kind='bar', color='lightcoral', edgecolor='black')
plt.title(f"Distribution of {col}", fontsize=14, weight='bold')
plt.ylabel("Count")
plt.xlabel(col)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()

# Save plot as JPG
filename = f"{output_folder}/{col.replace('/', '_').replace(' ', '_')}_c
plt.savefig(filename, format='jpg', dpi=300, bbox_inches='tight')
plt.close() # Close plot to free memory

```

```

In [19]: import rdflib
import requests
import openai
from urllib.parse import urlparse
from collections import defaultdict
import time

# Hardcoded Groq/OpenAI-compatible API configuration
openai.api_key = "gsk_90IkD6E71WgdrE4nR0ewWGdyb3FYwjY7DbT5SjqQc7Q2Ixxgac4y0"
openai.api_base = "https://api.groq.com/openai/v1"

# Property descriptions for relevance checks
property_prompts = {
    "hasBloodType": "is a valid bloodtype?",
    "hasAge": "is a valid age?",
    "belongsToHospitalSection": "is a valid hospital section?",
    "hasCity": "is a real city?",
    "hasCountry": "is a real country?",
    "hasDiseaseDiagnosis": "is a disease?",
    "hasEducationLevel": "is a degree?",
    "hasGeneralPractice": "is a person's name?",
    "hasHairColor": "is a color?",
    "hasIncome": "is an integer?",
    "hasMaritalStatus": "is marital status?",
    "hasName": "is a person's name?",
    "hasPostalCode": "is a valid postal code?",
    "hasStreet": "is a real street?",
    "isAllergicTo": "is a type of allergy?",
    "spokenIn": "is a country?"
}

# Load the RDF data
g = rdflib.Graph()
g.parse("results.ttl", format="ttl")

# Validate if a URI exists (returns True/False)
def is_uri_valid(uri):
    try:
        response = requests.head(uri, timeout=5)

```

```

        return response.status_code == 200
    except requests.exceptions.RequestException:
        return False

# Extract URIs grouped by property (predicate)
def extract_uris_by_property(graph):
    uris_by_predicate = defaultdict(set)
    for subj, pred, obj in graph:
        if isinstance(obj, rdflib.URIRef):
            pred_name = pred.split("#")[-1] if "#" in pred else pred.split("/")[-1]
            uris_by_predicate[pred_name].add(str(obj))
    return uris_by_predicate

# Relevance check using Groq LLM (OpenAI-compatible)
def check_relevance_with_ai(uri, prompt_text):
    parsed = urlparse(uri)
    entity = parsed.path.strip("/").split("/")[-1].replace("_", " ")
    prompt = f'"{entity}" {prompt_text} Answer only with True or False.'

    try:
        response = openai.ChatCompletion.create(
            model="meta-llama/llama-4-scout-17b-16e-instruct",
            messages=[
                {"role": "system", "content": "Answer only with True or False"},
                {"role": "user", "content": prompt}
            ],
            temperature=0
        )
        reply = response['choices'][0]['message']['content'].strip().lower()
        return reply.startswith("true")
    except Exception as e:
        print(f"Error checking relevance for {uri}: {e}")
        return False

# Run validity and relevance checks
def validate_and_check_all(graph):
    uris_by_property = extract_uris_by_property(graph)
    total_counts = {}

    for prop, uris in uris_by_property.items():
        print(f"\nChecking property: {prop}")
        prompt_text = property_prompts.get(prop, "is this a relevant entity?")
        valid_and_relevant_count = 0

        for uri in uris:
            valid = is_uri_valid(uri)
            relevant = check_relevance_with_ai(uri, prompt_text)
            print(f"URI: {uri}\n → Valid: {valid}, Relevant: {relevant}")

            if valid and relevant:
                valid_and_relevant_count += 1

        time.sleep(5) # Delay of 5 seconds after each request

    total_counts[prop] = valid_and_relevant_count
    print(f"✅ Count of Valid & Relevant URIs for {prop}: {valid_and_relevant_count}")

```

```
print("\n📊 Final Summary:")
for prop, count in total_counts.items():
    print(f"{prop}: {count} valid & relevant URIs")

# Execute
validate_and_check_all(g)
```

Checking property: spokenIn

URI: <https://dbpedia.org/page/Nigeria>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/India>

→ Valid: True, Relevant: True

URI: https://dbpedia.org/page/United_Kingdom

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/UK>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Italy>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Brazil>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/China>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/France>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/USA>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Spain>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Japan>

→ Valid: True, Relevant: True

✅ Count of Valid & Relevant URIs for spokenIn: 9

Checking property: takesMedication

URI: <https://dbpedia.org/page/Metformin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Isoniazid>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Artemisinin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Insulin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Salbutamol>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Lisinopril>

→ Valid: True, Relevant: True

✅ Count of Valid & Relevant URIs for takesMedication: 6

Checking property: hasDiseaseDiagnosis

URI: https://dbpedia.org/page/Diabetes_mellitus

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Hypertension>

→ Valid: True, Relevant: False

URI: <https://dbpedia.org/page/Malaria>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Tuberculosis>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Asthma>

→ Valid: True, Relevant: True

✅ Count of Valid & Relevant URIs for hasDiseaseDiagnosis: 3

Checking property: hasBloodType

URI: <https://www.wikidata.org/wiki/Q19831446>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q19831448>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q19831447>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q19831449>
→ Valid: True, Relevant: False
✅ Count of Valid & Relevant URIs for hasBloodType: 0

Checking property: belongsToHospitalSection
URI: <https://www.wikidata.org/wiki/Q56857412>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q56857410>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q56857411>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q56857413>
→ Valid: True, Relevant: False
✅ Count of Valid & Relevant URIs for belongsToHospitalSection: 0

Checking property: hasCountry
URI: <https://dbpedia.org/page/Nigeria>
→ Valid: True, Relevant: True
URI: <https://dbpedia.org/page/India>
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/United_Kingdom
→ Valid: True, Relevant: True
URI: <https://dbpedia.org/page/Brazil>
→ Valid: True, Relevant: True
URI: <https://dbpedia.org/page/France>
→ Valid: True, Relevant: True
URI: <https://dbpedia.org/page/USA>
→ Valid: False, Relevant: True
URI: <https://dbpedia.org/page/Japan>
→ Valid: True, Relevant: True
✅ Count of Valid & Relevant URIs for hasCountry: 6

Checking property: hasMaritalStatus
URI: <https://www.wikidata.org/wiki/Q3089373>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q3089372>
→ Valid: True, Relevant: False
URI: <https://www.wikidata.org/wiki/Q304>
→ Valid: False, Relevant: False
✅ Count of Valid & Relevant URIs for hasMaritalStatus: 0

Checking property: hasEducationLevel
URI: <https://www.dbpedia.org/members/bachelor-program>
→ Valid: False, Relevant: False
URI: <https://www.dbpedia.org/members/master-program>
→ Valid: False, Relevant: False
URI: <https://www.dbpedia.org/members/doctorate-program>
→ Valid: False, Relevant: False
✅ Count of Valid & Relevant URIs for hasEducationLevel: 0

Checking property: isAllergicTo
URI: https://dbpedia.org/page/Gluten_allergy

→ Valid: False, Relevant: False
URI: https://dbpedia.org/page/Soy_allergy
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/Dust_mite_allergy
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/Lactose_intolerance
→ Valid: True, Relevant: False
URI: https://dbpedia.org/page/Peanut_allergy
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/Pollen_allergy
→ Valid: False, Relevant: True
URI: https://dbpedia.org/page/Dust_allergy
→ Valid: False, Relevant: True
URI: https://dbpedia.org/page/Nickel_allergy
→ Valid: True, Relevant: True
✅ Count of Valid & Relevant URIs for isAllergicTo: 4

Checking property: type
URI: <http://example.org/Person>
→ Valid: False, Relevant: True
✅ Count of Valid & Relevant URIs for type: 0

Checking property: hasCity
URI: <https://dbpedia.org/page/Mumbai>
→ Valid: True, Relevant: True
URI: <https://dbpedia.org/page/Tokyo>
→ Valid: True, Relevant: True
URI: <https://dbpedia.org/page/Lagos>
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/New_York
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/Sao_Paulo
→ Valid: False, Relevant: True
URI: <https://dbpedia.org/page/Paris>
→ Valid: True, Relevant: True
✅ Count of Valid & Relevant URIs for hasCity: 5

📊 Final Summary:
spokenIn: 9 valid & relevant URIs
takesMedication: 6 valid & relevant URIs
hasDiseaseDiagnosis: 3 valid & relevant URIs
hasBloodType: 0 valid & relevant URIs
belongsToHospitalSection: 0 valid & relevant URIs
hasCountry: 6 valid & relevant URIs
hasMaritalStatus: 0 valid & relevant URIs
hasEducationLevel: 0 valid & relevant URIs
isAllergicTo: 4 valid & relevant URIs
type: 0 valid & relevant URIs
hasCity: 5 valid & relevant URIs

```
In [26]: import rdflib
import requests
import openai
from urllib.parse import urlparse
from collections import defaultdict
import time
```

```

# Hardcoded Groq/OpenAI-compatible API configuration
openai.api_key = "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"
openai.api_base = "https://api.groq.com/openai/v1"

# Property descriptions for relevance checks
property_prompts = {
    "hasBloodType": "is a valid bloodtype?",
    "hasAge": "is a valid age?",
    "belongsToHospitalSection": "is a valid hospital section?",
    "hasCity": "is a real city?",
    "hasCountry": "is a real country?",
    "hasDiseaseDiagnosis": "is a disease?",
    "hasEducationLevel": "is a reference to education degree?",
    "hasGeneralPractice": "is a person's name?",
    "hasHairColor": "is a color?",
    "hasIncome": "is an integer?",
    "hasMaritalStatus": "is a reference to marital status?",
    "hasName": "is a person's name?",
    "hasPostalCode": "is a valid postal code?",
    "hasStreet": "is a real street?",
    "isAllergicTo": "is a type of allergy?",
    "spokenIn": "is a country?"
}

# Load the RDF data
g = rdflib.Graph()
g.parse("results.ttl", format="ttl")

# Validate if a URI exists (returns True/False)
def is_uri_valid(uri):
    try:
        response = requests.head(uri, timeout=5)
        return response.status_code == 200
    except requests.exceptions.RequestException:
        return False

# Extract URIs grouped by property (predicate)
def extract_uris_by_property(graph):
    uris_by_predicate = defaultdict(set)
    for subj, pred, obj in graph:
        if isinstance(obj, rdflib.URIRef):
            pred_name = pred.split("#")[-1] if "#" in pred else pred.split("/")[-1]
            uris_by_predicate[pred_name].add(str(obj))
    return uris_by_predicate

# Relevance check using Groq LLM (OpenAI-compatible)
def check_relevance_with_ai(uri, prompt_text):
    parsed = urlparse(uri)
    entity = parsed.path.strip("/").split("/")[-1].replace("_", " ")
    prompt = f"'{entity}' {prompt_text} Answer only with True or False."

    try:
        response = openai.ChatCompletion.create(
            model="meta-llama/llama-4-scout-17b-16e-instruct",
            messages=[

```

```

        {"role": "system", "content": "Answer only with True or False"}
        {"role": "user", "content": prompt}
    ],
    temperature=0
)
reply = response['choices'][0]['message']['content'].strip().lower()
return reply.startswith("true")
except Exception as e:
    print(f"Error checking relevance for {uri}: {e}")
    return False

# Run validity and relevance checks
def validate_and_check_all(graph):
    uris_by_property = extract_uris_by_property(graph)
    total_counts = {}

    for prop, uris in uris_by_property.items():
        print(f"\nChecking property: {prop}")
        prompt_text = property_prompts.get(prop, "is this a relevant entity?")
        valid_and_relevant_count = 0

        for uri in uris:
            valid = is_uri_valid(uri)
            relevant = check_relevance_with_ai(uri, prompt_text)
            print(f"URI: {uri}\n → Valid: {valid}, Relevant: {relevant}")

            if valid and relevant:
                valid_and_relevant_count += 1

        time.sleep(5) # Delay of 5 seconds after each request

    total_counts[prop] = valid_and_relevant_count
    print(f"✅ Count of Valid & Relevant URIs for {prop}: {valid_and_relevant_count}")

print("\n📊 Final Summary:")
for prop, count in total_counts.items():
    print(f"{prop}: {count} valid & relevant URIs")

# Execute
validate_and_check_all(g)

```

Checking property: hasEducationLevel

URI: <https://www.wikidata.org/wiki/Q163727>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q183816>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q849697>

→ Valid: True, Relevant: False

✓ Count of Valid & Relevant URIs for hasEducationLevel: 0

Checking property: hasMaritalStatus

URI: <https://www.wikidata.org/wiki/Q28341938>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q179115>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q37713312>

→ Valid: True, Relevant: False

✓ Count of Valid & Relevant URIs for hasMaritalStatus: 0

Checking property: spokenIn

URI: <https://dbpedia.org/page/Nigeria>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/India>

→ Valid: True, Relevant: True

URI: https://dbpedia.org/page/United_Kingdom

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/UK>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Italy>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Brazil>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/China>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/France>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/USA>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Spain>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Japan>

→ Valid: True, Relevant: True

✓ Count of Valid & Relevant URIs for spokenIn: 7

Checking property: takesMedication

URI: <https://dbpedia.org/page/Metformin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Isoniazid>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Artemisinin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Insulin>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Salbutamol>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Lisinopril>

→ Valid: True, Relevant: True

✓ Count of Valid & Relevant URIs for takesMedication: 6

Checking property: hasBloodType

URI: <https://www.wikidata.org/wiki/Q19831454>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q19831453>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q19831449>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q19831455>

→ Valid: True, Relevant: False

✓ Count of Valid & Relevant URIs for hasBloodType: 0

Checking property: hasDiseaseDiagnosis

URI: https://dbpedia.org/page/Diabetes_mellitus

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Hypertension>

→ Valid: True, Relevant: False

URI: <https://dbpedia.org/page/Malaria>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Tuberculosis>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Asthma>

→ Valid: True, Relevant: True

✓ Count of Valid & Relevant URIs for hasDiseaseDiagnosis: 3

Checking property: belongsToHospitalSection

URI: <https://www.wikidata.org/wiki/Q56857412>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q56857410>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q56857411>

→ Valid: True, Relevant: False

URI: <https://www.wikidata.org/wiki/Q56857413>

→ Valid: True, Relevant: False

✓ Count of Valid & Relevant URIs for belongsToHospitalSection: 0

Checking property: hasCountry

URI: <https://dbpedia.org/page/Nigeria>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/India>

→ Valid: True, Relevant: True

URI: https://dbpedia.org/page/United_Kingdom

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/Brazil>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/France>

→ Valid: True, Relevant: True

URI: <https://dbpedia.org/page/USA>

→ Valid: False, Relevant: True

URI: <https://dbpedia.org/page/Japan>

→ Valid: True, Relevant: True

✓ Count of Valid & Relevant URIs for hasCountry: 6

Checking property: isAllergicTo

URI: https://dbpedia.org/page/Gluten_allergy

```
→ Valid: False, Relevant: False
URI: https://dbpedia.org/page/Soy_allergy
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/Dust_mite_allergy
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/Lactose_intolerance
→ Valid: True, Relevant: False
URI: https://dbpedia.org/page/Peanut_allergy
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/Pollen_allergy
→ Valid: False, Relevant: True
URI: https://dbpedia.org/page/Dust_allergy
→ Valid: False, Relevant: True
URI: https://dbpedia.org/page/Nickel_allergy
→ Valid: True, Relevant: True
✅ Count of Valid & Relevant URIs for isAllergicTo: 4
```

```
Checking property: type
URI: http://example.org/Person
→ Valid: False, Relevant: True
✅ Count of Valid & Relevant URIs for type: 0
```

```
Checking property: hasCity
URI: https://dbpedia.org/page/Mumbai
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/Tokyo
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/Lagos
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/New_York
→ Valid: True, Relevant: True
URI: https://dbpedia.org/page/Sao_Paulo
→ Valid: False, Relevant: True
URI: https://dbpedia.org/page/Paris
→ Valid: True, Relevant: True
✅ Count of Valid & Relevant URIs for hasCity: 5
```

```
📊 Final Summary:
hasEducationLevel: 0 valid & relevant URIs
hasMaritalStatus: 0 valid & relevant URIs
spokenIn: 7 valid & relevant URIs
takesMedication: 6 valid & relevant URIs
hasBloodType: 0 valid & relevant URIs
hasDiseaseDiagnosis: 3 valid & relevant URIs
belongsToHospitalSection: 0 valid & relevant URIs
hasCountry: 6 valid & relevant URIs
isAllergicTo: 4 valid & relevant URIs
type: 0 valid & relevant URIs
hasCity: 5 valid & relevant URIs
```

```
In [4]: from rdflib import Graph

# Load JSON-LD
g = Graph()
g.parse("results.ttl", format="turtle")
```

```
# Serialize to Turtle
g.serialize(destination="result.ttl", format="turtle")

print("Conversion complete: result.ttl")
```

Conversion complete: result.ttl

```
In [3]: ttl_content = """@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ex: <http://example.org/> .
@prefix dist: <http://example.org/distribution#> .

# Person Shape
ex:PersonShape a sh:NodeShape ;
  sh:targetClass ex:Person ;

  sh:property [
    sh:path ex:hasName ;
    sh:datatype xsd:string ;
    sh:minCount 1 ;
    dist:distribution "categorical" ;
    dist:categories (
      [ dist:value "Alice" ; dist:probability "0.3" ]
      [ dist:value "Bob" ; dist:probability "0.3" ]
      [ dist:value "Charlie" ; dist:probability "0.4" ]
    ) ;
  ] ;

  sh:property [
    sh:path ex:hasDateOfBirth ;
    sh:datatype xsd:date ;
    sh:minCount 1 ;
    dist:distribution "uniform" ;
    dist:startDate "1960-01-01"^^xsd:date ;
    dist:endDate "2010-12-31"^^xsd:date ;
  ] ;

  sh:property [
    sh:path ex:hasAddress ;
    sh:node ex:AddressShape ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    dist:distribution "node" ;
  ] ;

  sh:property [
    sh:path ex:knowsLanguage ;
    sh:node ex:LanguageShape ;
    sh:minCount 1 ;
    sh:maxCount 3 ;
    dist:distribution "categorical" ;
  ] ;

  sh:property [
    sh:path ex:hasDoctor ;
    sh:node ex:DoctorShape ;
```

```

        sh:minCount 1 ;
        dist:distribution "node" ;
    ] ;

    sh:property [
        sh:path ex:hasParent ;
        sh:node ex:PersonShape ;
        sh:maxCount 2 ;
        dist:distribution "node" ;
    ] ;

    sh:property [
        sh:path ex:hasInsurance ;
        sh:node ex:InsuranceShape ;
        sh:minCount 1 ;
        dist:distribution "node" ;
    ] ;

    sh:property [
        sh:path ex:hasSymptoms ;
        sh:datatype xsd:string ;
        sh:minCount 1 ;
        sh:maxCount 5 ;
        dist:distribution "categorical" ;
        dist:categories (
            [ dist:value "Fever" ; dist:probability "0.3" ]
            [ dist:value "Cough" ; dist:probability "0.4" ]
            [ dist:value "Fatigue" ; dist:probability "0.2" ]
            [ dist:value "Headache" ; dist:probability "0.1" ]
        ) ;
    ] ;

    sh:property [
        sh:path ex:hasDiagnosisDate ;
        sh:datatype xsd:dateTime ;
        sh:maxCount 1 ;
        dist:distribution "normal" ;
        dist:mean "2022-01-01T12:00:00"^^xsd:dateTime ;
        dist:stddev "30"^^xsd:float ; # days
    ] ;

    sh:property [
        sh:path ex:hasProfileURL ;
        sh:datatype xsd:anyURI ;
        sh:maxCount 1 ;
        dist:distribution "template" ;
        dist:template "https://example.org/profile/{uuid}" ;
    ] .

# Address Shape
ex:AddressShape a sh:NodeShape ;
sh:targetClass ex:Address ;
sh:property [
    sh:path ex:hasStreet ;
    sh:datatype xsd:string ;
    dist:distribution "categorical" ;

```



```

        dist:categories (
            [ dist:value "Main St" ; dist:probability "0.4" ]
            [ dist:value "High St" ; dist:probability "0.3" ]
            [ dist:value "Broadway" ; dist:probability "0.3" ]
        ) ;
    ] ;
    sh:property [
        sh:path ex:hasPostalCode ;
        sh:datatype xsd:string ;
        sh:pattern "[0-9]{5}$" ;
        dist:distribution "regex" ;
        dist:pattern "[0-9]{5}$" ;
    ] ;
    sh:property [
        sh:path ex:hasCity ;
        sh:datatype xsd:string ;
        dist:distribution "categorical" ;
        dist:categories (
            [ dist:value "New York" ; dist:probability "0.4" ]
            [ dist:value "Berlin" ; dist:probability "0.3" ]
            [ dist:value "Tokyo" ; dist:probability "0.3" ]
        ) ;
    ] ;
    sh:property [
        sh:path ex:hasCountry ;
        sh:in ("USA" "Germany" "India" "Japan") ;
        dist:distribution "categorical" ;
    ] .

# Doctor Shape
ex:DoctorShape a sh:NodeShape ;
sh:targetClass ex:Doctor ;
sh:property [
    sh:path ex:hasName ;
    sh:datatype xsd:string ;
    dist:distribution "categorical" ;
] ;
sh:property [
    sh:path ex:specializesIn ;
    sh:datatype xsd:string ;
    sh:in ("Cardiology" "Neurology" "Oncology" "General") ;
    dist:distribution "categorical" ;
] ;
sh:property [
    sh:path ex:worksAt ;
    sh:node ex:HospitalShape ;
    dist:distribution "node" ;
] .

# Hospital Shape
ex:HospitalShape a sh:NodeShape ;
sh:targetClass ex:Hospital ;
sh:property [
    sh:path ex:hasName ;
    sh:datatype xsd:string ;
    dist:distribution "categorical" ;

```

```

] ;
sh:property [
  sh:path ex:hasSection ;
  sh:datatype xsd:string ;
  sh:maxCount 3 ;
  dist:distribution "categorical" ;
] ;
sh:property [
  sh:path ex:hasAccreditationLevel ;
  sh:datatype xsd:string ;
  sh:in ("A" "B" "C") ;
  dist:distribution "categorical" ;
  dist:categories (
    [ dist:value "A" ; dist:probability "0.6" ]
    [ dist:value "B" ; dist:probability "0.3" ]
    [ dist:value "C" ; dist:probability "0.1" ]
  ) ;
] .

# Insurance Shape
ex:InsuranceShape a sh:NodeShape ;
sh:targetClass ex:Insurance ;
sh:property [
  sh:path ex:hasProvider ;
  sh:datatype xsd:string ;
  dist:distribution "categorical" ;
] ;
sh:property [
  sh:path ex:hasCoverageAmount ;
  sh:datatype xsd:float ;
  dist:distribution "normal" ;
  dist:mean "50000"^^xsd:float ;
  dist:stddev "15000"^^xsd:float ;
] ;
sh:property [
  sh:path ex:isGovernmentFunded ;
  sh:datatype xsd:boolean ;
  dist:distribution "bernoulli" ;
  dist:probabilityTrue "0.3"^^xsd:float ;
] .

# Language Shape
ex:LanguageShape a sh:NodeShape ;
sh:targetClass ex:Language ;
sh:property [
  sh:path ex:languageName ;
  sh:datatype xsd:string ;
  sh:in ("English" "Spanish" "German" "Japanese") ;
  dist:distribution "categorical" ;
] ;
sh:property [
  sh:path ex:spokenIn ;
  sh:datatype xsd:string ;
  sh:minCount 1 ;
  dist:distribution "categorical" ;
] .

```

```
.....
```

```
with open("personshape.ttl", "w", encoding="utf-8") as f:  
    f.write(ttl_content)
```

```
In [ ]: from rdflib import Graph, URIRef  
  
def convert_nt_to_ttl_with_replacement(input_file, output_file, defined_subj  
    g = Graph()  
  
    # Read .nt file line-by-line  
    with open(input_file, 'r') as f:  
        for line in f:  
            parts = line.strip().strip('.').split(' ', 2)  
            if len(parts) == 3:  
                original_subject, original_predicate, obj = parts  
                obj = obj.strip()  
  
                # Clean angle brackets if present  
                if obj.startswith('<') and obj.endswith('>'):  
                    obj = obj[1:-1]  
  
                # Replace subject and predicate  
                subject = URIRef(defined_subject)  
                predicate = URIRef(defined_predicate)  
                object_ = URIRef(obj) if obj.startswith('http') else obj  
  
                g.add((subject, predicate, URIRef(obj)))  
  
    # Serialize to TTL  
    g.serialize(destination=output_file, format='turtle')  
  
# Example usage  
convert_nt_to_ttl_with_replacement(  
    input_file='owl_type=parsed_sorted.nt',  
    output_file='owl_type=parsed_sorted.ttl',  
    defined_subject='http://example.org/Person',  
    defined_predicate='http://example.org/hasName'  
)
```

```
In [5]: from rdflib import Graph, URIRef, Literal, BNode  
import re  
  
def parse_object(obj_str):  
    """Parses RDF object: handles URIs, literals (typed, language-tagged, si  
    obj_str = obj_str.strip()  
  
    # Literal with datatype  
    if obj_str.startswith('\"') and '^<' in obj_str:  
        match = re.match(r'\"(.*)\"\\^<(.+)>', obj_str)  
        if match:
```

```

        value, datatype = match.groups()
        return Literal(value, datatype=URIRef(datatype))

    # Language-tagged literal (e.g., "text"@en)
    elif obj_str.startswith('"""') and '@' in obj_str:
        match = re.match(r'"""(.*)"@([a-zA-Z\-\+])', obj_str)
        if match:
            value, lang = match.groups()
            return Literal(value, lang=lang)

    # Simple literal
    elif obj_str.startswith('"""') and obj_str.endswith('"""'):
        return Literal(obj_str[1:-1])

    # URI
    elif obj_str.startswith('<') and obj_str.endswith('>'):
        return URIRef(obj_str[1:-1])

    # Blank node
    elif obj_str.startswith('_:'):
        return BNode(obj_str[2:]) # drop "._:" prefix

    raise ValueError(f"Unsupported object format: {obj_str}")

def convert_nt_to_ttl_with_replacement(input_file, output_file, defined_subj
g = Graph()

    with open(input_file, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line or not line.endswith('.'):
                continue
            parts = line[:-1].strip().split(' ', 2)
            if len(parts) != 3:
                continue

            _, _, obj_str = parts
            subject = URIRef(defined_subject)
            predicate = URIRef(defined_predicate)
            obj = parse_object(obj_str)

            g.add((subject, predicate, obj))

    g.serialize(destination=output_file, format='turtle')

# Example usage
convert_nt_to_ttl_with_replacement(
    input_file='owl_type=parsed_sorted.nt',
    output_file='owl_type=parsed_sorted.ttl',
    defined_subject='http://example.org/Person',
    defined_predicate='http://example.org/hasName'
)

```

```

In [11]: from rdflib import Literal, URIRef, BNode
import re

```

```

def parse_object(obj_str):
    """Parses RDF object."""
    obj_str = obj_str.strip()

    # Literal with datatype
    if obj_str.startswith('\"') and '^<' in obj_str:
        match = re.match(r'\"(.*)\"\\^\\<(.+)>', obj_str)
        if match:
            value, datatype = match.groups()
            return Literal(value, datatype=URIRef(datatype))

    # Language-tagged literal
    elif obj_str.startswith('\"') and '@' in obj_str:
        match = re.match(r'\"(.*)\"@([a-zA-Z\\-]+)', obj_str)
        if match:
            value, lang = match.groups()
            return Literal(value, lang=lang)

    # Simple literal
    elif obj_str.startswith('\"') and obj_str.endswith('\"'):
        return Literal(obj_str[1:-1])

    # URI
    elif obj_str.startswith('<') and obj_str.endswith('>'):
        return URIRef(obj_str[1:-1])

    # Blank node
    elif obj_str.startswith('_:'):
        return BNode(obj_str[2:])

    raise ValueError(f"Unsupported object format: {obj_str}")

def replace_last_segment(uri, new_segment):
    """Replace the last segment after '/' or '#' in the URI with new_segment
    if '#' in uri:
        base, _ = uri.rsplit('#', 1)
        return f"{base}#{new_segment}"
    else:
        base, _ = uri.rsplit('/', 1)
        return f"{base}/{new_segment}"

def modify_nt_file(input_path, output_path):
    with open(input_path, 'r', encoding='utf-8') as infile, \
        open(output_path, 'w', encoding='utf-8') as outfile:

        for line in infile:
            line = line.strip()
            if not line or not line.endswith('.'):
                outfile.write(line + '\n')
                continue

            parts = line[:-1].strip().split(' ', 2) # remove trailing '.' t
            if len(parts) != 3:
                outfile.write(line + '\n')
                continue

```

```

subj = parts[0]
pred = parts[1]
obj = parts[2]

# Replace last segment of subject URI if applicable
if subj.startswith('<') and subj.endswith('>'):
    subj_uri = subj[1:-1]
    new_subj = f"<{replace_last_segment(subj_uri, 'person')}>"
else:
    new_subj = subj

# Replace last segment of predicate URI if applicable
if pred.startswith('<') and pred.endswith('>'):
    pred_uri = pred[1:-1]
    new_pred = f"<{replace_last_segment(pred_uri, 'hasDisease')}>"
else:
    new_pred = pred

# Write the modified triple, keeping object as is
new_line = f"{new_subj} {new_pred} {obj} .\n"
outfile.write(new_line)

# Example usage:
modify_nt_file('owl_type=parsed_sorted.nt', 'output.nt')

```